EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments

Jundong Xu · Qingchuan Li · Jiaying Wu · Yihuai Lan · Shuyue Stella Li · Huichi Zhou · Bowen Jiang · Lei Wang · Jun Wang · Anh Tuan Luu · Caiming Xiong · Hae Won Park · Bryan Hooi · Zhiyuan Hu

ABSTRACT

Large language model (LLM) agents have achieved strong performance on a wide range of benchmarks, yet most evaluations assume static environments. In contrast, real-world deployment is inherently dynamic, requiring agents to continually align their knowledge, skills, and behavior with changing environments and updated task conditions. To address this gap, we introduce ***EvoArena****, a benchmark suite that models environment changes as sequences of progressive updates across terminal, software, and social domains. We further propose* ***EvoMem****, a patch-based memory paradigm that records memory evolution as structured update histories, enabling agents to reason about environmental evolution through changes in their memory. Experiments show that current agents struggle on EvoArena, achieving an average accuracy of 39.6% across evolving terminal, software, and social-preference domains. EvoMem consistently improves performance, yielding an average gain of 1.5% on EvoArena and also improving standard benchmarks such as GAIA and LoCoMo by 6.1% and 4.8%. Beyond individual tasks, EvoMem further improves chain-level accuracy by 3.7% on EvoArena, where success requires completing a consecutive sequence of related evolutionary subtasks. Mechanistic analysis shows that EvoMem improves evidence capture in the memory, indicating better preservation of complete evolving environment states. Our results highlight the importance of modeling evolution in both evaluation and memory for reliable agent deployment.*

1 Introduction

LLM agents are increasingly expected to operate in external environments such as web systems, terminal workspaces, software repositories, and personalized assistants. Modern agent systems combine reasoning [25], tool use [26], and memory to execute multi-step procedures [24], act on environment state, and adjust their behavior within a task episode, achieving strong performance on benchmarks for web navigation [27], software engineering [7], tool use [14], and general agentic reasoning [14]. A harder setting arises when the same agent must remain useful across versions of the same environment, where interfaces, rules, code states, and user preferences continue to

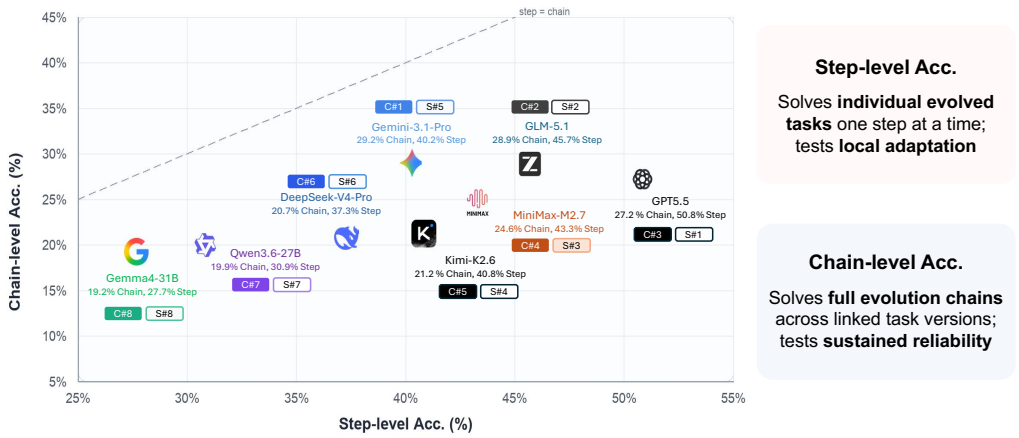

FIGURE 1 **Step accuracy vs. chain accuracy on EvoArena.** The closer to the upper-right corner the better.

change.

Yet this progress leaves a central deployment challenge underexplored: **agents are usually evaluated on static environment snapshots.** In most benchmarks, the interface, rules, task distribution, and success criteria are fixed once the benchmark is constructed [7, 11, 14, 27]. Recent dynamic evaluations improve freshness or interaction realism through refreshed tasks, asynchronous events, or self-evolving instances [3, 9, 22], but they rarely test persistent environment evolution, where the same setting changes across versions. In deployment, APIs, workflows, codebases, and user preferences continually evolve. A reliable agent must know what changed, what still holds, and how to act under the current version.

To address this gap, we introduce **EvoArena**, a benchmark suite for evaluating agents under **persistent environment evolution**. EvoArena organizes each environment into a chain of **progressively evolving releases**, where the same underlying goal or setting is preserved while interfaces, rules, workflows, code states, or user preferences change over time. This formulation turns environment evolution into a measurable capability: an agent must solve the current task, identify which updates matter, and avoid reusing behaviors tied to obsolete versions. Concretely, EvoArena includes **Terminal-Bench-Evo** for evolving terminal workflows, **SWE-Chain-Evo** for evolving codebases, and **PersonaMem-Evo** for evolving user preferences. Together, these benchmarks evaluate both *forward adaptation* to new changes and *version compatibility* with still-valid prior knowledge, testing whether agents remain reliable beyond static snapshots.

Using EvoArena, we find that even strong agent systems degrade substantially in evolving environments. One recurring failure mode is **state collapse**: most memory-based agents maintain memory as *a single latest state*, such as a retrieved memory bank or episodic store [2, 24, 26]. This design is effective when newer information safely supersedes older information, but becomes brittle when different environment versions require different behaviors. A workflow permission update, for instance, may overwrite an earlier rule that still applies to an older release, a different organization, or a future rollback. The agent then loses both the previous behavior and the context explaining when it was valid. This reveals a need for **version-aware state tracking**: agents must retain the latest memory, recover relevant prior states, and reason over why each

update occurred.

To address these limitations, we introduce **EvoMem**, a lightweight git-like memory paradigm for evolving environments. EvoMem augments a standard memory system with an append-only patch history that records meaningful memory changes. Each patch stores the pre-update memory, post-update memory, update rationale, and supporting evidence from the triggering context. This makes memory evolution *traceable*: agents can inspect what changed, why it changed, and which evidence justified the update. At inference time, the agent retrieves from the latest memory by default, while selectively retrieving relevant patches when a query depends on overwritten states, conflicting evidence, or earlier environment versions. In this way, EvoMem turns memory updates into a compact evidence trail for reasoning over environment evolution.

Empirically, **EvoMem improves agent robustness across both EvoArena and standard long-horizon agent benchmarks**, with an average 1.5% performance gain observed across agents and backbone models. In addition, EvoMem improves chain-level accuracy by 3.7%, showing its ability to support agents in completing sequences of related tasks under a continuous environment evolving. Beyond aggregate performance, our analysis shows why patch histories help: on PersonaMem-Evo, EvoMem yields stronger gains on temporal trajectory and multi-pattern synthesis questions, where agents must retain evolving and dispersed preference evidence. It also improves row-level evidence capture, indicating that patches better preserve complete preference states required for downstream reasoning. These findings suggest a broader implication: reliable agents in changing environments should treat memory as an evolving history of grounded updates, keeping the latest state connected to the prior states, rationales, and evidence that produced it. Overall, we make the following contributions::

- We introduce **EvoArena**, a benchmark suite that evaluates agents across progressively evolving workflow, software, and social environments.
- We propose **EvoMem**, a patch-based memory paradigm that preserves traces of environmental evolution, enabling agents to reason more robustly in evolving environments.
- We show that existing agents struggle under environment evolution, while EvoMem consistently improves robustness and evidence retention across diverse settings.

2 Related Work

Dynamic and Evolving Agent Benchmarks. Agent benchmarks increasingly evaluate realistic interaction settings, including web navigation [27], software engineering [7], tool use and general agentic reasoning [11, 14], device control [19], terminal workflows [13], enterprise tasks [1], and personalized memory [6]. Recent dynamic benchmarks improve evaluation freshness or interaction realism through refreshed software tasks, asynchronous events, or generated benchmark variants [3, 9, 22]. Long-horizon personalization benchmarks further study how agents use evolving user context, but often introduce preference change as a single update rather than a multi-step version history [10]. EvoArena instead evaluates persistent environment evolution: the same setting changes across versions, requiring agents to adapt to new releases while preserving behavior that remains valid. We instantiate this setting by extending terminal,

Benchmark	Domain	What Evolves?	PE	IC	CE
WebArena [27]	Web	Static tasks	✗	✗	✗
SWE-bench [7]	Software	Static issues	✗	✗	✗
GAIA [14]	Tool / Web	Static tasks	✗	✗	✗
AgentBench [11]	Multi-env.	Static tasks	✗	✗	✗
AndroidWorld [19]	Device control	Static tasks	✗	✗	✗
Terminal-Bench [13]	Terminal	Static tasks	✗	✗	✗
WorkArena++ [1]	Workflow	Static tasks	✗	✗	✗
PersonaMem [6]	Personalization	User memory	△	△	✗
SWE-bench-Live [9]	Software	Task refresh	✗	✗	✗
GAIA2 [3]	Tool / Web	Async events	△	✓	✗
Benchmark Self-Evolving [22]	General	Generated tasks	✗	✗	✗
HorizonBench [10]	Personalization	Preference changes	△	✓	✗
EvoArena	**Multi-domain**	**Dynamic Env**	✓	✓	✓

TABLE 1 **Comparison of EvoArena with related agent benchmarks.** PE denotes persistent environment evolution, where the same environment or user state evolves across versions; IC denotes implicit change, where the agent must infer or handle changes from context or interaction; CE denotes chain evaluation, where success is measured over temporally linked task sequences. ✓denotes supported, ✗denotes not supported, and △denotes partial support.

enterprise workflow, software-engineering, and personalization benchmarks into evolving chains.

Self-Evolving Agents and Memory. Self-evolving agents improve their own behavior through reflection [20], skill accumulation [21], reusable skill refinement [26], or scaffold adaptation [23]. These methods focus on agent-side improvement, while EvoArena studies changes in the external environment. Memory systems such as structured long-term memory [24], production-scale persistent memory [2], and memory-based agent frameworks [8] continuously update stored knowledge. However, they usually consolidate memory toward the latest state, which can obscure overwritten states and the reasons behind updates. EvoMem treats memory updates as evidence, preserving what changed, why it changed, and which context supports each version-dependent behavior.

3 EvoArena: Benchmarking Agents under Persistent Environment Evolution

Real-world agent deployment often occurs in **evolving environments**, where interfaces, rules, code states, user preferences, and system constraints change over time. Agents must infer these changes from observations, interaction feedback, and task outcomes, then decide which prior knowledge remains valid under the current version. Existing benchmarks largely evaluate agents on static snapshots, leaving this version-aware capability underexplored.

To study this setting, we construct **EvoArena**, a benchmark suite that evaluates agents under persistent environment evolution. As shown in Figure 2, EvoArena covers three representative

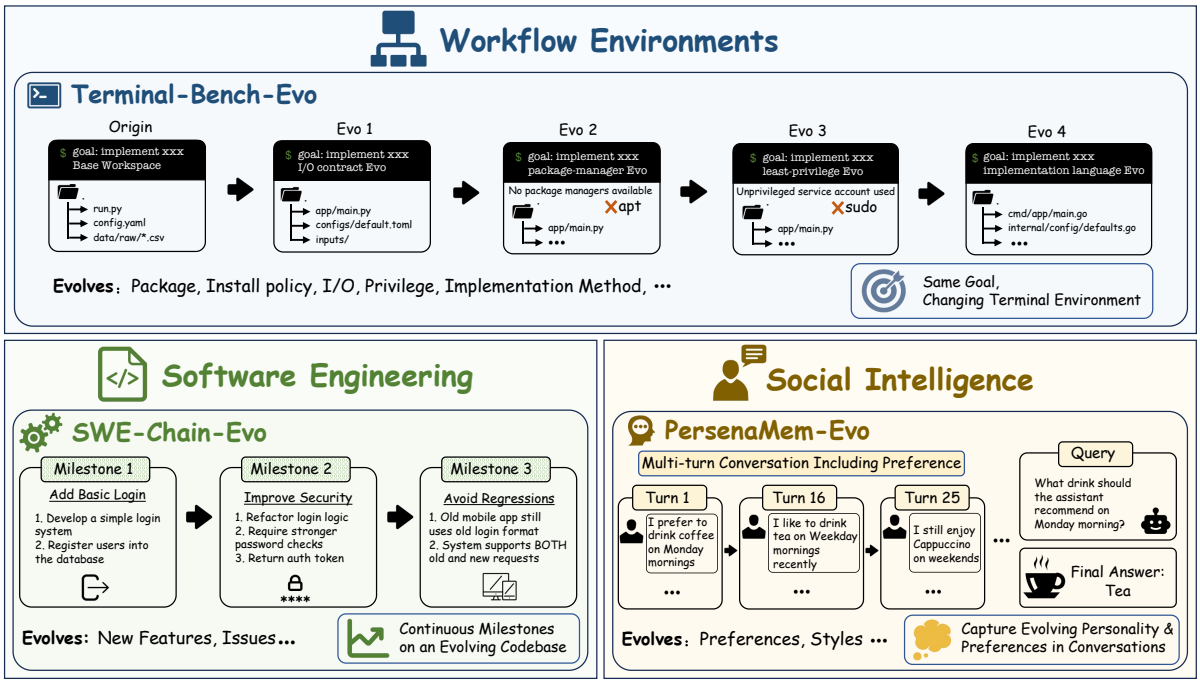

FIGURE 2 **EvoArena construction.** We convert static agent benchmarks into versioned evolution chains across executable workflows, software engineering, and social intelligence, testing whether agents can adapt to new changes while preserving still-valid prior behavior.

evolution regimes: **(1) executable workflow evolution** (Terminal-Bench-Evo), **(2) software evolution** (SWE-Chain-Evo), and **(3) preference evolution** (PersonaMem-Evo). These domains capture complementary forms of non-stationarity: terminal workflows change through dependency, interface, path, and validation updates; codebases evolve through continuous implementation milestones; and user preferences shift across long-horizon conversations. Together, they provide a unified testbed for evaluating whether agents can track environment changes, adapt to new versions, and preserve still-valid prior behavior.

Figure 3 summarizes the distribution of domain and question across EvoArena, and Table 2 summarizes the key statistics for each subset. We describe each subset below and point each construction detail to the corresponding appendix subsubsection.

3.1 Executable Workflow Evolution: Terminal-Bench-Evo

We choose executable terminal workflows because they expose the concrete surfaces through which deployed agents experience environment drift: files move, command-line interfaces change, dependencies are upgraded, validation scripts tighten, and policy constraints become part of the runnable task. We define an *evolving workflow* as a discrete version chain: an ordered sequence of executable releases $\{v^{(1)}, \ldots, v^{(m)}\}$ that preserve the same Terminal-Bench objective [13] but change the surrounding instruction, environment, files, dependencies, interfaces, or tests between releases. Each version is a standalone terminal episode with a fixed container/workspace; the

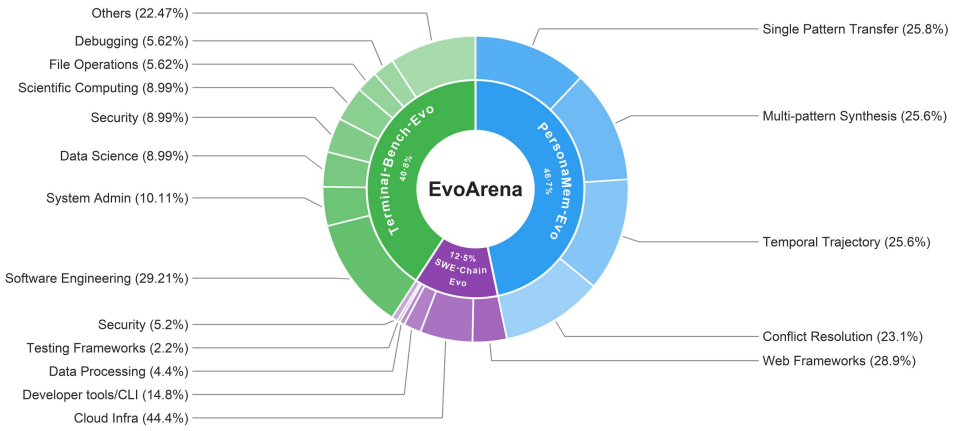

FIGURE 3 **Distribution of EvoArena**. The central circle shows domain proportions, and the surrounding panels show the question type distribution within each domain.

environment does not change during that episode. Our construction follows five stages that mirror the subsections below. First, **workflow-state analysis** extracts the objective, environment, files, dependencies, interfaces, and validation rules of each task. Second, **evolution-plan design** specifies realistic updates over mutable workflow components while preserving the underlying task objective. Then, **inherited version realization** materializes these updates as executable releases, where later versions inherit the realized environment of earlier ones. This is followed by **quality control and oracle validation**, which filters or repairs releases to ensure that each version is executable, internally consistent, and solvable by the reference solution. Finally, **benchmark assembly, metadata, and metrics** groups validated releases into workflow chains and records the metadata and evaluation units used for step- and chain-level scoring. We describe each stage in detail below.

(1) Workflow-state analysis. For each original task, we first convert the raw Terminal-Bench instance into a structured workflow state containing its objective, executable environment, relevant files, dependencies, interface/I/O contracts, and validation rules. This representation identifies what should remain stable across the chain, such as the user-facing objective, and what can plausibly evolve, such as paths, invocation commands, package versions, or test expectations. The resulting state becomes the input to evolution planning. Appendix E.1.2 specifies the workflow-state fields and the mutable execution assumptions extracted from each task.

(2) Evolution-plan design. Using the structured state, we design a sequence of candidate updates that preserve the objective while changing the operational procedure. The update taxonomy covers I/O and protocol changes, CLI/API changes, dependency or toolchain updates, workspace and module refactoring, and semantic or policy changes. For example, a later version may move the required output path, replace an invocation flag, upgrade a runtime, reorganize source modules, add a permission constraint, or tighten the validation rule. The output is an ordered plan specifying which workflow component changes at each version. Appendix E.1.3 defines the evolution taxonomy and gives the component-level change categories used during planning.

(3) Inherited version realization. We then instantiate the evolution plan in chronological order. Each version is produced by jointly editing the instruction, container environment, files/configuration, reference solution, tests, and metadata. Crucially, version t starts from the realized environment of version $t-1$, so earlier path, dependency, permission, interface, and validation changes persist unless the current update explicitly revises them. This inheritance makes the chain a coherent workflow history rather than a set of independent variants, while still leaving each version as a complete executable task. Appendix E.2.1 describes how inherited environments, instructions, reference solutions, and tests are materialized for each release.

(4) Quality control and oracle validation. After realization, we verify that each version is executable, internally consistent, and solvable. The reference solution must pass the version-specific tests, the instruction/environment/tests must agree with one another, and later versions must preserve inherited changes from earlier versions. Versions with ambiguous instructions, incomplete dependencies, inconsistent tests, or broken reference solutions are repaired or removed. This turns candidate releases into validated executable benchmark instances. Appendix E.2.2 details the consistency checks and filtering criteria, while Appendix E.2.3 reports the oracle-solution validation procedure.

Benchmark assembly, evaluation, and metadata. Finally, we assemble the retained versions into workflow chains and store chain position, update type, changed components, environment summaries, validation summaries, and reference-solution information for analysis. The executable unit is one *versioned task*: the agent receives a fixed version-specific instruction and container/workspace, and is scored by that version's tests. We report both *step accuracy*, averaged over versioned tasks, and *chain accuracy*, where a chain is correct only if all of its versions are solved. Appendix E.2.4 lists the stored metadata and dataset statistics, and Appendix E.1.1 defines the step- and chain-level metrics.

The final Terminal-Bench-Evo dataset contains 89 initial Terminal-Bench tasks, from which 356 evolved task versions are constructed as five-version workflow chains; after quality control removes 4 invalid versions, this results in 352 evolved versioned tasks and 441 total task instances including the initial versions. Chains contain four to five versions, with mean length 4.96 and median length 5. The evolved versioned tasks cover diverse operational changes, with I/O or protocol changes forming the largest group (49.1%), followed by workspace/module/staging changes (13.4%), CLI/API changes (10.5%), dependency/toolchain/capability changes (8.0%), and semantic, policy, or evaluation-rule changes (4.6%). The benchmark also spans a broad range of terminal domains, with top categories including software engineering (29.5%), system administration (10.0%), security (9.1%), data science (9.1%), and scientific computing (8.6%). In terms of difficulty, most instances are medium or hard, with 268 medium, 152 hard, 20 easy, and 1 expert task. These statistics characterize Terminal-Bench-Evo as executable workflow evolution. An example is shown in Figure 4.

3.2 Software Evolution: SWE-Chain-Evo

We choose software repositories because the codebase itself is an evolving environment: earlier API changes, dependency updates, tests, and implementation decisions remain active when later

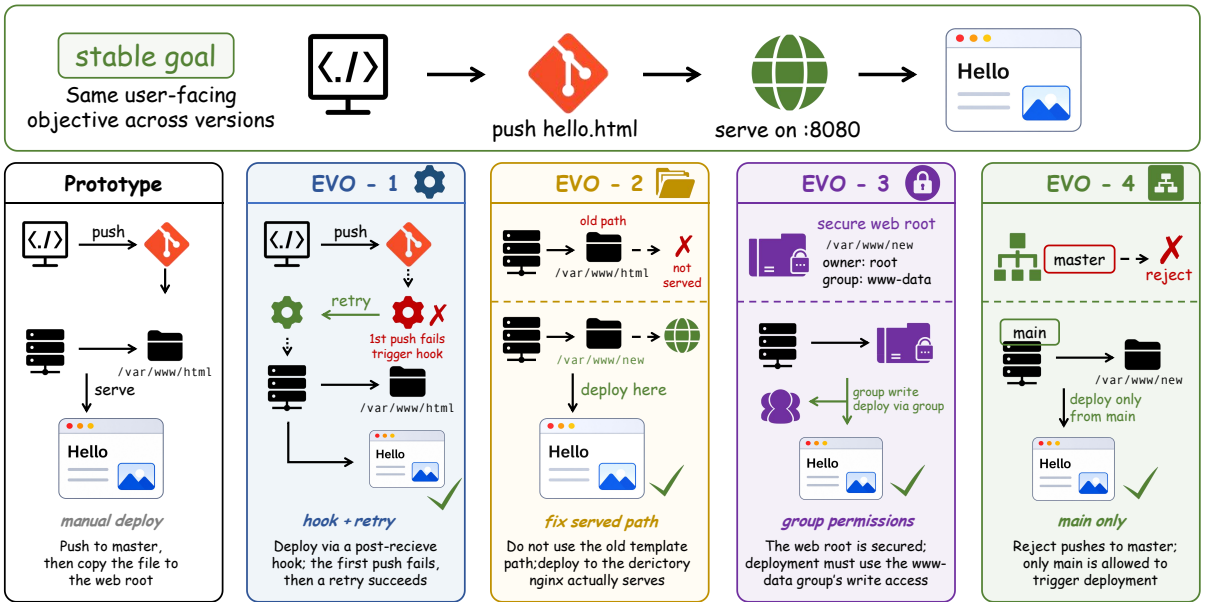

FIGURE 4 **An example of Terminal-Bench-Evo.** The end goal stays fixed: push `hello.html` and serve it on port 8080, while each version changes a key operational constraint: deployment mechanism, served path, permissions, or branch policy.

requirements arrive. We define software evolution as a chronological chain of *milestone releases* $\{m^{(1)}, \ldots, m^{(T)}\}$ within a repository window. Each milestone is a localized, testable development objective implemented by one or more related commits. The history is embedded in the repository state: at step t, the agent sees the current accumulated repository snapshot and a milestone instruction, produces a patch, and is evaluated on tests for that milestone. Our construction follows six stages. First, **repository selection** identifies active, diverse, and testable repositories. Second, **update-window extraction** selects contiguous chronological commit ranges. Then, **milestone grouping** clusters related commits into coherent development objectives. Next, **task-description synthesis** converts each milestone into a SWE-bench-style requirement. This is followed by **Docker evaluation construction**, which builds executable tests and validates reference solutions. Finally, **chronological chain assembly** orders validated milestones into evolution chains through oracle repository-state progression. Each stage is described in detail below.

(1) Repository selection. We start from candidate GitHub repositories and retain projects with recent development activity, non-trivial size, reproducible dependency installation, and automated tests that can be run in a Docker environment. We also require domain diversity across web frameworks, infrastructure, observability, security, developer tools, data processing, and testing frameworks. This defines the repository pool in which software evolution is both realistic and executable. Appendix E.3.1 gives the repository filtering criteria and domain coverage.

(2) Update-window extraction. For each retained repository, an update window is a contiguous chronological range of commits from which candidate evolution chains can be formed.

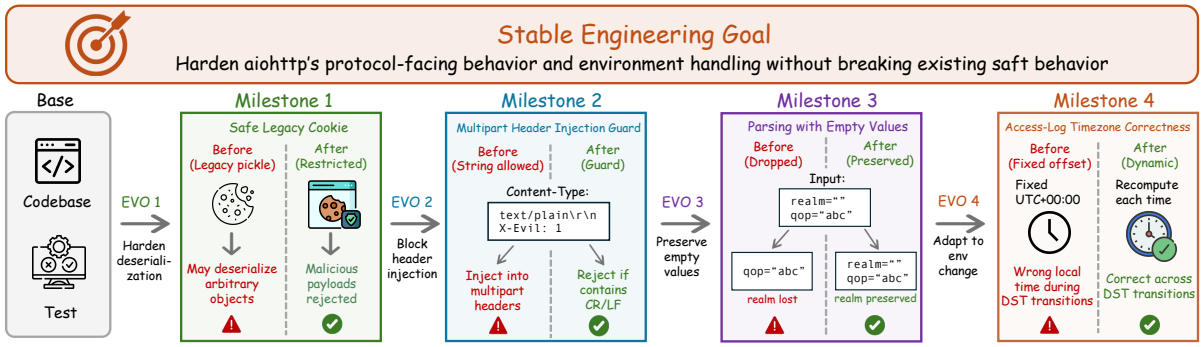

FIGURE 5 **Example evolution chain in SWE-Chain-Evo.** The `aiohttp` chain evolves across four milestones, progressively hardening protocol and environment boundaries while preserving existing compatible behavior.

This keeps later milestones grounded in the same repository history rather than mixing unrelated snapshots, and it provides the local temporal context for grouping commits into milestones. Appendix E.3.2 defines update windows and describes how commit ranges are selected before milestone grouping.

(3) Milestone grouping. Within each update window, we group nearby commits into milestones using commit-message semantics and manual inspection of code diffs. A valid milestone must implement one coherent objective, such as a feature addition, bug fix, API adjustment, dependency migration, or test-backed maintenance update. This sits between two less useful extremes: release notes often aggregate many unrelated changes, while individual commits can be too small, incomplete, refactoring-only, or noisy. Formatting-only edits, incidental dependency bumps, merge artifacts, and unrelated changes are removed from the candidate milestone patch when they do not contribute to the objective, and the cleaned milestone is kept only if it remains executable and test-valid. Appendix E.3.2 also describes commit grouping, noisy-change removal, and milestone coherence checks.

(4) Task-description synthesis. Each validated milestone is converted into a SWE-bench-style instance. The task description is written from pre-milestone context and natural development signals such as commit messages, issue or PR text when available, tests, and high-level code-change summaries. We review descriptions to avoid leaking the reference patch, exact code edits, changed files, or solution-specific implementation details unless such information would naturally appear in a user requirement. The agent's expected output is a repository patch, which is applied to the pre-milestone chain state before evaluation. Appendix E.4.1 specifies the allowed description signals, leakage controls, and patch-output format.

(5) Docker evaluation construction. We build a Docker environment for each instance, install dependencies, configure build and test commands, and validate that the reference milestone patch succeeds in that environment. Fail-to-Pass tests are tests that fail on the pre-milestone state and pass after applying the reference patch; every retained milestone must have at least one such test. Pass-to-Pass tests are regression tests that pass both before and after the reference patch, and are included when stable tests are available. A step is resolved only if the agent patch applies, all selected Fail-to-Pass tests pass, and no selected Pass-to-Pass regression test fails.

Appendix E.4.2 defines the test-selection rules, Docker validation, and resolution criterion.

Benchmark assembly, evaluation, and metadata. Validated milestones are ordered by their original chronological commit order within each repository window. After step t is evaluated, we apply the *reference* milestone update, not the agent's predicted patch, to form the repository state for step $t+1$. This oracle-state progression isolates adaptation to evolving repository states from compounding errors caused by earlier failed agent patches. Thus the benchmark tests whether agents can solve each localized requirement under the current accumulated codebase; memory-based agents may additionally use prior interaction context, but repository-state accumulation is fixed by the reference history. When the same underlying milestone appears in overlapping chain contexts, it contributes multiple task-step slots but one unique milestone. We evaluate agents using two metrics: *Step Accuracy*, which measures whether an agent solves each individual milestone at its chain position, and *Chain Accuracy*, defined as the number of consecutively solved milestones from the beginning of a chain before the first unsolved or unexecuted milestone, divided by the total chain length. Appendix E.4.3 describes chronological ordering, oracle-state progression, and the distinction between task-step slots and unique milestones.

Overall, SWE-Chain-Evo contains 50 evolution chains from 12 repositories, with 493 chain-step instances and 145 unique repository milestones. A *chain-step instance* is a milestone evaluated at a particular position within an evolution chain, and serves as the unit on which agents are evaluated; since some underlying repository updates appear in multiple chain contexts, deduplicating by milestone identity yields 145 unique milestones. Chains contain 5–15 steps, with mean length 9.86 and median length 10, including 10 five-step chains, 9 seven-step chains, 10 ten-step chains, 7 twelve-step chains, 7 thirteen-step chains, and 7 fifteen-step chains. Milestones are focused but non-trivial: each milestone is represented as one real commit transition, modify 2.72 files on average, with 38.6% modifying multiple files, and have a median gold patch size of 53 diff lines. The benchmark spans Go (404 instances, 81.9%) and Python (89 instances, 18.1%).

Each milestone instance includes at least one Fail-to-Pass test, with 7.13 Fail-to-Pass tests on average and a median of 2; Pass-to-Pass tests have mean 25.85 and median 4, with no instances lacking Pass-to-Pass tests. Besides, the chains exhibit substantial cross-step dependency: among the 443 non-initial milestone instances, 29.8% modify at least one file touched by an earlier milestone, and 14.2% modify a file touched by the immediately preceding step. This reflects real software evolution, where later milestones often depend on and revise earlier implementation decisions. Compared with independent SWE-bench-style instances, SWE-Chain-Evo evaluates whether agents can solve localized software requirements under an evolving repository state where previous API, dependency, test, and implementation changes remain active. Appendix E.4.4 reports detailed statistics, and an example is shown in Figure 5.

3.3 Social Intelligence Evolution: PersonaMem-Evo

We choose long-horizon preference interactions because personalization agents must infer user-specific behavior from implicit evidence, and real preferences often change as routines, constraints, experiences, or priorities shift. We define preference evolution as an ordered trajectory of latent

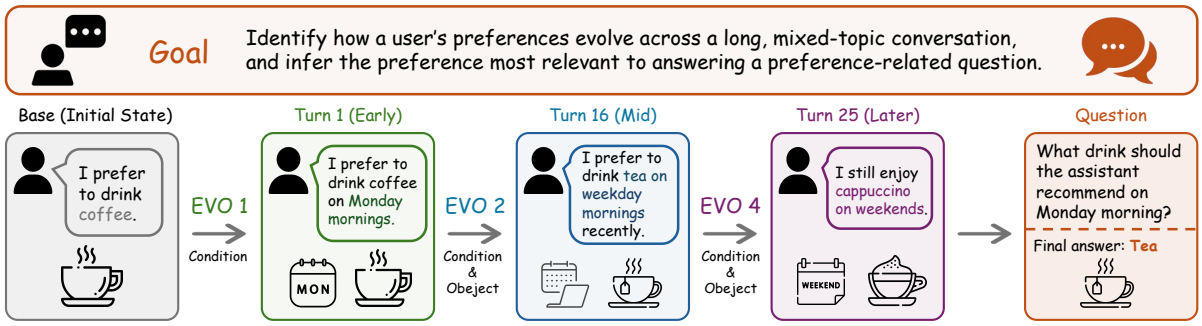

FIGURE 6 **Example evolution chain in PersonaMem-Evo.** The conversation history reveals evolving user preferences, and the query requires matching the relevant condition to the most appropriate current preference.

preference states $\{p^{(0)}, \ldots, p^{(k)}\}$ for the same user and preference dimension. Earlier preference states remain in the interaction history as historically grounded evidence, while later episodes may revise their strength, scope, style, temporal validity, or applicable conditions. The evaluation unit is one multiple-choice question paired with a long mixed-topic history: the agent must select the answer best supported by the interaction history, not merely by persona stereotypes or commonsense priors. Our construction follows six stages. First, **seed persona expansion and preference cleaning** creates structured profiles and cleaned preference inventories. Second, **implicit interaction history construction** expresses preferences through realistic multi-turn interactions. Then, **temporal preference evolution** introduces ordered preference trajectories. Next, **preference-inference question generation** creates questions that require generalizing observed behavior to new settings. This is followed by **dual-blind filtering and answer-option construction**, which removes shortcuts and builds balanced options. Finally, **benchmark assembly, metadata, and evaluation** produces the final benchmark instances with metadata and task-accuracy evaluation. The following sections examine each stage in detail.

(1) Seed persona expansion and preference cleaning. Built on PersonaMem-v2 [6], we sample seed personas from PersonaHub [4] and expand each seed into a structured profile with demographic, occupational, lifestyle, personality, and social-context fields. From this profile, we synthesize categorized preferences, including stereotypical, anti-stereotypical, neutral, therapy-related, and health/medical preferences. We then remove duplicated, empty, or inconsistent entries and assign topic labels. This defines the stable user background and the initial preference inventory that later interaction episodes will express or revise. Appendix E.5.1 describes seed expansion, preference categories, cleaning rules, and topic labeling.

(2) Implicit interaction history construction. For each preference record, we generate one or more short conversation segments between a user with a AI-assistant in which the preference is implied through the user's request, constraints, wording, repeated behavior, or surrounding context. These segments are instantiated from a fixed set of interaction formats, such as asking the assistant to revise an email, translate a message, answer a knowledge question, give advice about a personal problem, or polish a social-media post. Each segment is associated with the preference record that generated it; the record specifies the preference text, its source category (e.g., stereotypical, anti-stereotypical, neutral, health-related, or therapy-related),

whether it describes the user or another person, and whether it supersedes or deletes an earlier preference. We concatenate the generated segments into mixed-topic long-context histories while preserving segment boundaries, so the agent must recover user-specific evidence from realistic conversational context. Appendix E.6.6 provides the detailes of the interaction format and the preference record.

(3) Temporal preference evolution. Each updated state is instantiated as a new implicit interaction episode and annotated with its previous state, updated state, change family, step index, and full trajectory. For example, a user may first prefer intense hiking, later avoid it after an injury, and then prefer light trail walks after recovery. This design requires agents to distinguish the latest preference from outdated but previously valid states. Appendix E.6.1 defines the change families, trajectory sampling, update annotations, and temporal-state construction.

(4) Preference-inference question generation. From the long histories, we generate questions that require applying observed behavioral patterns to new decision settings. The question types are single-pattern transfer, multi-pattern synthesis, conflict resolution, and temporal trajectory prediction. These categories test whether agents can transfer an uncommon preference to a new domain, combine dispersed preference signals, arbitrate between competing preferences, or extrapolate a preference trajectory. Appendix E.6.2 describes the definition of each question type, generation prompts, and complexity levels.

(5) Dual-blind filtering and answer-option construction. Each candidate question is packaged as a balanced four-way multiple-choice item. Distractors are designed to be plausible and may include stereotype-consistent choices, semantically related alternatives, or outdated preference states. We then apply dual-blind filtering: a question is rejected if it can be answered from the persona profile alone or without using the interaction history. This removes shortcuts and makes the retained questions depend on conversational memory instead of demographic priors or answer-option artifacts. Appendix E.6.3 defines the persona-only and no-context filters, and Appendix E.6.4 describes distractor construction and answer-option balancing.

Benchmark assembly, evaluation, and metadata. Finally, each accepted question is paired with its long-context history and metadata, including persona conversation, topic label, scenario type, reasoning type, complexity level, and, when applicable, preference-change family and trajectory step. To construct preference-evolution chains, we group preferences within the same persona conversation as belonging to the same evolving chain when their cosine similarity exceeds 70% in chronological order. We evaluate agents using two metrics: *Step Accuracy*, which measures accuracy on each individual multiple-choice question, and *Chain Accuracy*, where a chain is counted as correct only if all questions in the corresponding preference-evolution chain are answered correctly. Appendix E.6.5 lists the benchmark fields, metadata schema, and evaluation format.

Overall, PersonaMem-Evo contains 10 persona-level conversations and 505 preference-inference questions, averaging 50.5 questions per persona. It has four balanced question types : single-pattern transfer (130), multi-pattern synthesis (129), temporal trajectory prediction (129), and conflict resolution (117). Appendix E.6.7 reports the definition of each question type. The dataset is built with different difficulty level, with 120 easy (L1), 186 medium (L2), and 199 hard questions (L3).

Terminal-Bench-Evo	
Statistics	**Numbers**
Init. tasks / chains	89
Constructed evol.	356
Invalid removed	4
Evolved tasks	352
Total inst.	441
Chain length	4.96 / 5
Len. range	4–5
Difficulty	
Easy	20
Medium	268
Hard	152
Expert	1

SWE-Chain-Evo	
Statistics	**Numbers**
Repositories	12
Evol. chains	50
Unique miles.	145
Total inst.	493
Avg commits / mile.	1.00
Chain steps	
Mean	9.86
5-step / 7-step	10 / 9
10-step / 12-step	10 / 7
13-step / 15-step	7 / 7
Avg P2P tests	25.85
Avg F2P tests	7.13

PersonaMem-Evo	
Statistics	**Numbers**
Persona conv.	10
Pref. chains	313
Total inst.	505
Avg conv. length	174.7K
Avg turns	597
Q / persona	50.5
Q type counts	130/129/129/117
Temp. src.	6 / 6.2
Difficulty	
Easy	120
Medium	186
Hard	199

TABLE 2 **Key statistics of the three EvoArena subsets.** Chain length reports mean / median. P2P and F2P denote Pass-to-Pass and Fail-to-Pass tests.

To make the benchmark useful for diagnosing memory-agent failures, we further annotate each question with its *source preferences*, i.e., the gold observed preference evidence required to answer correctly. This measures how many memories must be retrieved and combined: single-pattern transfer uses one source preference, conflict resolution uses two, multi-pattern synthesis uses three, while temporal trajectory questions require 2-10 source preferences with median 6 and mean 6.2, making them a direct test of long-chain preference tracking. Besides, the benchmark requires reasoning over long interaction histories: each persona-level chat history contains a median of 597 messages, with a median length of 174.7K tokens, testing whether agents can track evolving preferences from sparse signals distributed across long conversations. These statistics allow us to distinguish failures in memory retrieval, multi-evidence aggregation, stereotype bias, preference updating, and implicit preference extraction. Appendix E.6.7 reports detailed statistics, and an example is shown in Figure 6.

4 EvoMem: Patch-Based Memory Evolution

4.1 Overview: Memory as an Evolution Trace

Existing agent memory systems typically maintain a single latest memory state. This design is effective when newer observations safely supersede older ones. In evolving environments, however, knowledge is often *version-dependent*: a rule updated for a new workflow release may overwrite an earlier rule that remains valid for an older release, another organization, or a future rollback. Updating memory toward the latest state can therefore erase useful prior behavior and the context explaining when that behavior applied.

EvoMem addresses this limitation by augmenting an existing memory system with an explicit trace of memory evolution. As shown in Figure 7, EvoMem has two components. First, *patch*

recording stores meaningful non-additive memory updates, including what changed, why it changed, and which evidence triggered the update. Second, *patch-augmented retrieval* retrieves relevant historical patches together with the latest memory when a query depends on overwritten states, temporal changes, or version-specific behavior. This design keeps the base memory system intact while making memory updates inspectable and reusable for downstream reasoning.

4.2 Recording Memory Evolution as Patches

The goal of patch recording is to preserve the *behaviorally meaningful transitions* that occur when an agent updates its memory. In evolving environments, the latest memory content is often insufficient by itself: the agent may also need to know which earlier state was revised, what triggered the revision, and when the earlier state may still be valid. EvoMem therefore records non-additive memory updates as explicit patches, while leaving the base memory updater unchanged.

Let x_t denote the input observation at time t, and let M_{t-1} be the memory maintained by the base agent before observing x_t. The base memory updater produces a new memory state

$$M_t = U(M_{t-1}, x_t),$$

where U can be any agent-specific update function, such as updating a text memory bank, revising a user profile, modifying a memory graph, or refining a skill file.

EvoMem does not replace this updater. It monitors the transition from M_{t-1} to M_t and captures memory changes that would otherwise be discarded. We compute

$$\Delta_t = \mathrm{Diff}(M_{t-1}, M_t),$$

where Δ_t summarizes the affected memory entries and changed fields. EvoMem creates a patch only for *non-additive* updates, namely updates that revise, overwrite, or reinterpret existing memory. Purely additive observations can remain in the base memory without requiring a patch.

For each non-additive update, EvoMem writes a patch to an append-only patch history:

$$p_t = \left(\tau_t, C_t^-, C_t^+, r_t, z_t, e_t\right).$$

Here, τ_t denotes temporal metadata such as turn, session, or timestamp; C_t^- and C_t^+ denote the affected memory content before and after the update; r_t records the update rationale; z_t provides a concise semantic summary of the change; and e_t stores supporting evidence such as the triggering interaction, task context, execution feedback, or environment snapshot.

The resulting patch history is

$$\mathcal{P}_{1:t} = \{p_1, \ldots, p_t\}.$$

Thus, M_t stores the latest consolidated memory, while $\mathcal{P}_{1:t}$ preserves the intermediate transitions that explain how the memory reached its current state. This separation turns memory from a single mutable store into a versioned evolution trace: the agent can act from the latest memory while retaining access to prior states, rationales, and evidence for version-aware reasoning.

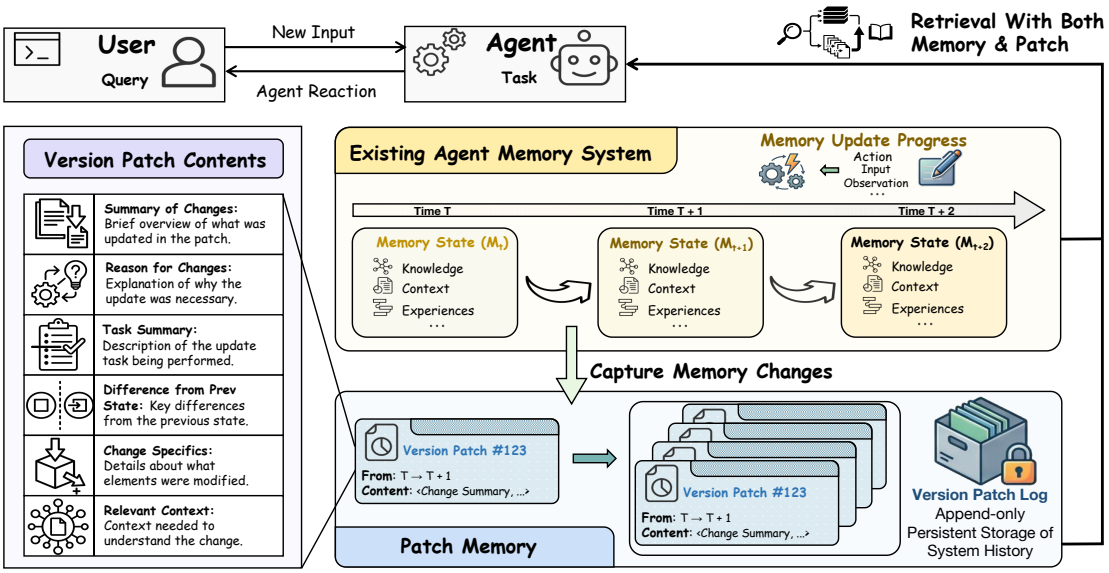

FIGURE 7 **Overview of EvoMem.** EvoMem augments a base memory system with an append-only patch history that records behaviorally meaningful memory updates and retrieves relevant patches as versioned evidence at inference time.

4.3 Retrieving Versioned Memory Evidence

Patch recording makes memory evolution available, but the agent still needs a way to use this history at inference time. The goal of patch-augmented retrieval is to expose version-relevant evidence only when it helps resolve the query. In ordinary cases, the latest memory may be sufficient. When the query depends on temporal changes, overwritten information, or update rationales, the agent should also access the patch history that explains how the current memory state was reached.

Given a query q, EvoMem first follows the standard memory-agent pipeline and retrieves evidence from the latest memory:

$$c_{\text{mem}} = R_{\text{mem}}(q, M_T),$$

where M_T is the final memory after processing all observations, and R_{mem} is the base retriever.

EvoMem then retrieves relevant patches from the patch history:

$$\mathcal{P}_q = R_{\text{patch}}(q, \mathcal{P}_{1:T}),$$

where R_{patch} returns the top-k patches relevant to q. These patches provide versioned evidence about previous memory states, the updates that changed them, and the rationales behind those updates.

The final context is

$$c(q) = \text{Concat}\left(c_{\text{mem}}, \mathcal{P}_q\right).$$

This context is provided to the agent for response generation or action selection. The latest memory supplies the current consolidated state, while retrieved patches explain how that state

changed and which prior states may still be relevant. As a result, the agent can ground its decision in both the current memory and the evolution trace behind it.

4.4 Instantiation Across Agents

EvoMem is a general memory abstraction with agent-specific schemas. Different agents represent and update knowledge in different forms, so each instantiation specifies three components: the base memory state M_T, the non-additive updates that should trigger patches, and the retrieval function over patch history. Across all cases, EvoMem follows the same principle: monitor memory updates, preserve meaningful overwritten states, and retrieve historical context when version-specific reasoning is required.

- **Terminus2 [13].** Terminus2 is a terminal-native agent without a persistent memory module. We instantiate M_T as distilled task-solving knowledge from prior terminal trajectories. Patches record strategy changes caused by evolving terminal environments, such as updated dependencies, interfaces, paths, or validation rules. Appendix F.1 details the chain-scoped terminal memory, transition-patch schema, retrieval-time memory construction, and anti-copying safeguards.
- **OpenHands [23].** We instantiate M_T as distilled software-engineering context from previous trajectories, including files, symbols, constraints, and execution outcomes. Patches record superseded implementation strategies, revised logic, and the observed cause of the change, helping the agent reuse prior debugging evidence while avoiding regressions. Appendix F.2 details the feature-level patch records, semantic-structural retrieval score, prompt construction, and the non-invasive integration with the OpenHands loop.
- **A-Mem [24].** A-Mem represents memory as semantically organized notes and links. We instantiate patches at the level of note and relation updates, preserving the previous note state, the revised content, and the rationale behind the change. This supports reasoning over evolving user preferences and temporally grounded memory conflicts. Appendix F.4 details the graph-diff trigger, patch representation, patch indexing, query-time retrieval, and patch-conditioned answer generation.
- **Memento-Skill [26].** Memento-Skill maintains a global `TIP.md` file as reusable skill memory. We treat `TIP.md` as M_T and create patches whenever it is revised, storing the task-specific tip update, triggering failure, and update rationale. This allows the agent to retrieve task-specific corrections that may be diluted in the consolidated skill file. Appendix F.3 details the versioned tip-memory family, store-time tip update, hybrid retrieval, prompt injection, and post-failure patching flow.

These instantiations show that EvoMem does not require a single memory format. It provides a lightweight interface for turning memory updates into reusable versioned evidence across terminal, software-engineering, conversational-memory, and skill-memory agents. Further implementation details are provided in Appendix F.

5 Experiments

5.1 Experimental Setup

We evaluate whether EvoMem improves agent robustness under environment evolution and long-horizon memory use. Our primary evaluation uses **EvoArena**, which contains three evolving benchmark subsets: Terminal-Bench-Evo, SWE-Chain-Evo, and PersonaMem-Evo. These subsets capture complementary forms of environment evolution: executable workflow changes, accumulated codebase changes, and evolving user preferences.

To test whether EvoMem remains useful beyond explicitly evolving benchmarks, we also evaluate on two standard agent benchmarks: GAIA [14] and LoCoMo [12]. We report the main task-level metric for each benchmark: task accuracy for EvoArena, LLM-judge accuracy for GAIA, and exact match for LoCoMo. Together, these benchmarks cover terminal interaction, software engineering, tool use, and long-horizon conversational memory.

Agents and models. We pair each benchmark with an agent suited to its interaction setting. For GAIA, we use Memento-Skill [26], which supports complex tool-use tasks. For LoCoMo and PersonaMem-Evo, we use A-Mem [24], which maintains long-horizon conversational memory. For Terminal-Bench-Evo and SWE-Chain-Evo, we use coding and system-interaction agents, including Terminus2 [13] and OpenHands [23]. We evaluate across multiple backbone models, including GPT-5.4-mini [16], GPT-5.5 [17], Gemini-3.1-Pro [5], Qwen3.6-27B [18], and Kimi-K2.6 [15]. Additional implementation details are provided in Appendix G.

Evaluation. We report *Step Accuracy* for all benchmarks, where a step denotes the basic evaluation unit of each benchmark: a versioned terminal task in *Terminal-Bench-Evo*, a milestone task in *SWE-Chain-Evo*, a multiple-choice preference question in *PersonaMem-Evo*, and a standard task/question instance in *GAIA* and *LoCoMo*. For EvoArena, we additionally report *Chain Accuracy*, which measures whether the agent completes an entire evolution chain without any mistake. Specifically, a *Terminal-Bench-Evo* chain is correct only if all versions derived from the same initial task are solved. A *SWE-Chain-Evo* chain accuracy measures how many milestones are consecutively solved from the beginning of the repository evolution chain before the first unsolved or unexecuted milestone, divided by the total number of milestones in the chain. Lastly a *PersonaMem-Evo* chain is correct only if all questions grouped into the same preference-evolution chain are answered correctly.

5.2 Main Results

Table **??** summarizes the main results and we highlight three findings.

Current agents struggle under persistent environment evolution, especially at the chain level. Across EvoArena, base agents achieve only moderate step-level performance, with average accuracies of 43.6% on Terminal-Bench-Evo, 29.2% on SWE-Chain-Evo, and 46.5% on PersonaMem-Evo. Performance drops further under the corresponding chain-level metrics for each subset: base agents achieve only 21.5% on Terminal-Bench-Evo, 10.6% on SWE-Chain-Evo, and 39.1% on PersonaMem-Evo. These results show that strong agent

systems still have difficulty tracking evolving execution conditions, accumulated codebase states, and shifting user preferences. More importantly, the substantially lower chain-level performance indicates that solving isolated steps does not necessarily translate into reliability across persistent environment evolution.

EvoMem improves robustness across evolving and standard settings. Adding EvoMem consistently improves performance across all EvoArena subsets. On average, EvoMem improves Terminal-Bench-Evo by 2.4%, SWE-Chain-Evo by 0.5%, and PersonaMem-Evo by 1.8%. These gains indicate that preserving memory evolution helps across different forms of environmental change, from terminal interface and validation shifts to software milestones and preference updates. EvoMem also improves performance on standard agent benchmarks, with average gains of 6.1% on GAIA and 4.8% on LoCoMo. Across all evaluated settings, EvoMem yields an average improvement of 2.6% at step-level and 3.7% at chain-level. These results suggest a broader design implication: robust agent memory should preserve update history as retrievable evidence, especially when task conditions change over time.

EvoMem improves even more at the chain level than at the step level. EvoMem yields larger gains under chain-level evaluation than under step-level evaluation, highlighting its value for maintaining consistency across dependent task sequences. On Terminal-Bench-Evo, the average improvement increases from 2.4% at the step level to 6.1% at the chain level. On SWE-Chain-Evo, the gain increases from 0.5% step accuracy to 2.9% chain accuracy, and on PersonaMem-Evo, from 1.8% step accuracy to 3.0% chain accuracy. This pattern suggests that EvoMem is especially effective when success requires agents to preserve and reuse evolution-aware knowledge over multiple related environment states.

6 Analysis: When and Why Does EvoMem Help?

Beyond aggregate performance, we analyze when and why EvoMem helps agents in evolving environments. We examine its behavior across domains as follows.

6.1 Terminal-Bench-Evo Mechanism Analysis

We perform an observational mechanism analysis to understand how EvoMem changes Terminus2's behavior on Terminal-Bench-Evo. The central hypothesis is that patch memory helps only when the agent operationalizes the retrieved transition information: it must notice what changed from previous variants, preserve the useful part of the old procedure, and modify the part that is no longer valid. We therefore analyze not only whether patch memory is available, but whether its contents appear in the agent's later reasoning or executed commands.

We design four factors for this purpose. *Patch example retrieval* tests whether EvoMem supplies at least one explicit EvoMem transition example, which describes a prior requirement, the evolved requirement, and the observed adaptation between them. *Evolved-requirement coverage* tests whether changed requirements summarized in EvoMem patch reappear in later agent's reasoning or commands, measuring whether the current EVO constraints become salient. *Patch uptake* tests whether terms from retrieved transition EvoMem patches reappear in subsequent reasoning

or commands, measuring whether the historical adaptation pattern is incorporated into the agent's plan. *Command-level patch uptake* is stricter: it only counts overlap with executed shell commands, and therefore measures whether patch information affects concrete terminal actions. For each factor, we compare baseline and EvoMem accuracy on the same subset of instances.

EvoMem helps when retrieved transitions are operationalized. The results support the operationalization hypothesis, as shown in Table **??**. When EvoMem retrieves no explicit patch example, the gain over baseline is 3.1%; when a patch example is retrieved, the gain rises to 6.5%. High evolved-requirement coverage also yields a larger gain than low coverage (5.3% vs. 2.1%), indicating that attending to the current variant's changed requirement is important for avoiding stale reuse. The largest gap appears for patch uptake: when the agent does not reuse any patch-transition terms, the gain is 2.6%, but when patch uptake is nonzero, the gain increases to 8.3%. Qualitatively, this pattern suggests that EvoMem is not simply adding more context. Terminal-Bench-Evo tasks often preserve much of the earlier procedure while changing a decisive requirement, such as an interface detail, file location, artifact convention, or environment constraint. EvoMem helps when the retrieved patch identifies this local transition and the agent carries it into reasoning or commands: the reusable part of the old strategy is preserved, while the part invalidated by the current evolution is revised.

6.2 SWE-Chain-Evo Mechanism Analysis

To understand why EvoMem improves SWE-Chain-Evo, we analyze whether it helps agents preserve behavior introduced by earlier milestones while solving the current code-editing task. This targets a central challenge in software evolution: implementing new requirements without regressing previously correct behavior. We therefore focus on *`PASS_TO_PASS` failures*, where at least one preservation test that should continue passing after the current milestone fails after the agent patch. This metric directly captures whether the agent breaks historical behavior.

EvoMem reduces regressions across backbones. Table 6 shows that EvoMem consistently reduces `PASS_TO_PASS` failure rates across all evaluated backbones. The average regression rate drops from 9.09% to 6.32%, with the largest reduction on Kimi-K2.6 (3.81%). This indicates that EvoMem helps agents preserve historical code constraints while adapting to new milestonesg. In SWE-Chain-Evo, where later edits build on earlier repository states, this regression reduction indicates that EvoMem helps agents maintain previously established behavior during software evolution.

6.3 PersonaMem-Evo Mechanism Analysis

To understand when and why EvoMem improves agent performance, we analyze whether it preserves the evolving evidence needed for downstream reasoning. PersonaMem-Evo provides fine-grained annotations of preference states and evolution trajectories, and its base agent, A-Mem, is explicitly memory-based. This setting allows us to study how EvoMem changes memory construction while minimizing confounds from tool-use or coding-specific behaviors.

EvoMem helps most when reasoning requires temporal or dispersed evidence.

Table 7 breaks down performance by question type and difficulty. EvoMem improves overall accuracy from 40.5% to 42.5%, with the largest gains on *temporal trajectory* and *multi-pattern synthesis* questions, both improving by 5.2%. This pattern matches the intended role of patch memory. Temporal trajectory questions require tracking how a preference changes over time, while multi-pattern synthesis requires retaining multiple preference signals scattered across the interaction history. In both cases, a single consolidated memory state may lose intermediate evidence, while EvoMem can recover relevant historical states from patch histories. The gains are less uniform on *conflict resolution* and *single-pattern transfer*, where performance decreases slightly. These categories require more than recovering missing evidence: conflict resolution requires inferring priority among competing preferences, and single-pattern transfer requires applying an uncommon or counter-stereotypical preference to a new setting. This suggests that patch histories improve evidence availability, while the final reasoning step remains a bottleneck for harder preference-inference cases.

Patch histories improve complete evidence preservation. To test the mechanism more directly, we measure whether the constructed memory store preserves ground-truth preference evidence. We report *clause-level capture*, which checks whether each atomic preference clause appears in at least one memory unit, and *row-level capture*, which requires all clauses needed for an example to be preserved. Table **??** shows that EvoMem improves clause-level capture from 89.4% to 90.3% and row-level capture from 72.5% to 74.9%. The larger row-level gain indicates that EvoMem better preserves complete preference states needed for downstream reasoning. Category-level results further support this explanation: the largest row-level capture gains occur on *temporal trajectory* (4.4%) and *multi-pattern synthesis* (3.5%), which are also the categories with the strongest accuracy gains. These results suggest that EvoMem improves performance by preserving coherent versioned evidence, especially when the answer depends on how preferences evolve across interactions.

6.4 Efficiency–Accuracy Trade-off

PersonaMem-Evo Efficiency. Figure 8(a) compares model accuracy against total token usage on PersonaMem-Evo, using token consumption as a proxy for inference efficiency. The results show that higher token usage does not reliably translate into higher accuracy. Gemma-4-31B-it achieves the best accuracy (52.6%) while using fewer tokens than the cross-model average (27.1M vs. 28.8M), and Kimi K2.6 reaches similarly strong accuracy (51.5%) with the lowest token usage among the evaluated models (24.5M). GLM 5.1 also attains high accuracy (51.5%) but requires above-average token usage (30.5M). By contrast, GPT-5.4-Mini consumes more tokens (35.9M) while obtaining lower accuracy (40.5%), and GPT-5.5 further amplifies this pattern, using 69.2M tokens while achieving 40.3% accuracy. These results show that longer trajectories do not necessarily translate into better task performance. Overall, PersonaMem-Evo demonstrates that efficiency and accuracy should be evaluated jointly rather than treating token budget as a substitute for capability.

Terminal-Bench-Evo Efficiency. Figure 8(b) also reports the efficiency–accuracy trade-off on Terminal-Bench-Evo, where token budgets are substantially larger than in PersonaMem-Evo.

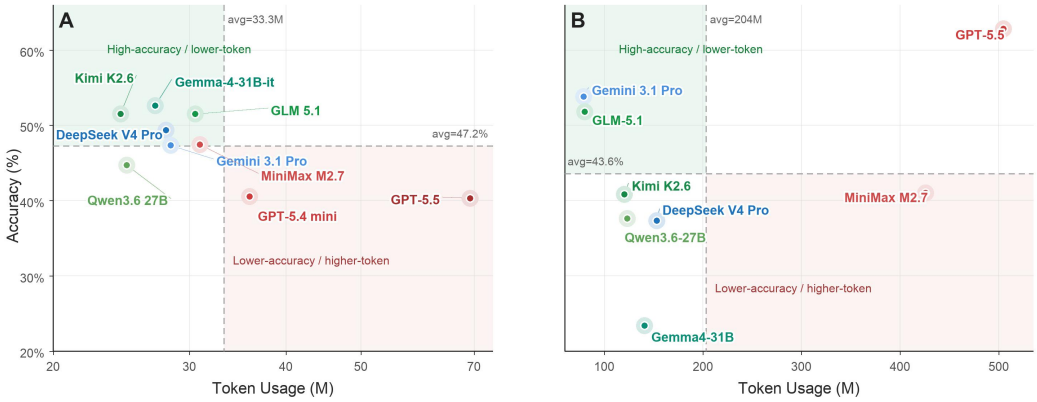

FIGURE 8 **Accuracy versus total token usage across evaluated backbone models.** Total token usage is measured in millions of tokens, and lower usage indicates higher inference efficiency. Dashed lines mark the cross-model averages.

GPT-5.5 achieves the highest terminal accuracy (62.8%) but consumes by far the most tokens (505.0M), indicating that its strong performance comes with a substantial inference cost. In contrast, Gemini 3.1 Pro and GLM-5.1 achieve strong accuracy (53.8% and 51.8%, respectively) while remaining well below the terminal average token usage (79.2M and 80.3M vs. 203.6M). Kimi K2.6 and Qwen3.6-27B also use below-average token budgets (120.6M and 123.4M), but their accuracies remain below the terminal average of 43.6%. Meanwhile, MiniMax M2.7 consumes a very large token budget (425.8M) while achieving only 41.0% accuracy, and Gemma4-31B obtains the lowest terminal accuracy (23.4%) despite moderate token usage (141.0M). These results further confirm that larger token budgets alone do not guarantee stronger task performance, making joint evaluation of accuracy and inference efficiency essential.

7 Conclusion

In this paper, we introduce EvoArena, a benchmark for evaluating LLM agents in environments where tasks, codebases, workflows, and user preferences evolve over time. We show that current agents remain limited under such non-stationary conditions, and propose EvoMem, a patch-based memory paradigm that records memory changes and their update context. Across EvoArena and standard agent benchmarks, EvoMem improves performance and better preserves evolving evidence. We hope this work supports future research on agents that can track changes, recover historical context, and adapt reliably in real-world evolving settings.

References

[1] Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier de Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. Workarena++: Towards compositional planning and reasoning-based common

knowledge work tasks. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

[2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *ECAI 2025 - 28th European Conference on Artificial Intelligence, 25-30 October 2025, Bologna, Italy - Including 14th Conference on Prestigious Applications of Intelligent Systems (PAIS 2025)*, pages 2993–3000, 2025.

[3] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Menard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Grégoire Mialon, and Thomas Scialom. Gaia2: Benchmarking LLM agents on dynamic and asynchronous environments. In *The Fourteenth International Conference on Learning Representations*, 2026.

[4] Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. Scaling synthetic data creation with 1,000,000,000 personas. *arXiv preprint arXiv:2406.20094*, 2024.

[5] Google. Gemini 3.1 pro: A smarter model for your most complex tasks. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/, 2026.

[6] Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, et al. Personamem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*, 2025.

[7] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[8] LangChain. Langgraph documentation. https://docs.langchain.com/oss/python/langgraph, 2024. Accessed: 2026-05-05.

[9] Kenan Li, Rongzhi Li, Linghao Zhang, Qirui Jin, Liao Zhu, Xiaosong Huang, Geng Zhang, Yikai Zhang, Shilin He, Chengxing Xie, et al. Repolaunch: Automating build&test pipeline of code repositories on any language and any platform. *arXiv preprint arXiv:2603.05026*, 2026.

[10] Shuyue Stella Li, Bhargavi Paranjape, Kerem Oktar, Zhongyao Ma, Gelin Zhou, Lin Guan, Na Zhang, Sem Park, Lin Chen, Diyi Yang, et al. Horizonbench: Long-horizon personalization with evolving preferences. *arXiv preprint arXiv:2604.17283*, 2026.

[11] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao

Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024.

[12] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pages 13851–13870, 2024.

[13] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026.

[14] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations*, 2024.

[15] Moonshot AI. Kimi k2.6. https://www.kimi.com/ai-models/kimi-k2-6, 2026.

[16] OpenAI. Introducing gpt-5.4 mini and nano. https://openai.com/index/introducing-gpt-5-4-mini-and-nano/, 2026.

[17] OpenAI. Introducing gpt-5.5. https://openai.com/index/introducing-gpt-5-5/, 2026.

[18] Qwen Team. Qwen3.6-27b: Flagship-level coding in a 27b dense model. https://qwen.ai/blog?id=qwen3.6-27b, 2026.

[19] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. In *The Thirteenth International Conference on Learning Representations*, 2025.

[20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[21] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

[22] Siyuan Wang, Zhuohan Long, Zhihao Fan, Xuanjing Huang, and Zhongyu Wei. Benchmark self-evolving: A multi-agent framework for dynamic LLM evaluation. In *Proceedings of the International Conference on Computational Linguistics*, pages 3310–3328, 2025.

[23] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025.

[24] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.

[26] Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, et al. Memento-skills: Let agents design agents. *arXiv preprint arXiv:2603.18743*, 2026.

[27] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

List of Appendices

A Limitations and Future Directions

EvoArena and EvoMem take a first step toward evaluating and improving LLM agents in evolving environments. Our benchmark focuses on three representative forms of environment evolution: executable workflow changes, software repository evolution, and long-horizon preference shifts. These domains are intentionally chosen because they expose different version-aware capabilities, including adapting to updated interfaces, reasoning over accumulated codebase changes, and preserving temporally grounded user preferences.

This scope also highlights the broader significance of the problem. Environment evolution

is a general challenge for deployed agents, and similar dynamics arise in robotics, embodied interaction, scientific workflows, multi-agent collaboration, and other long-running systems. Extending EvoArena to these settings would enable the community to study how agents track physical state changes, scientific protocol updates, collaborative dependencies, and evolving multi-agent roles. We view these extensions as an important direction for building agents that remain reliable across changing real-world conditions.

B Broader Impact

This work studies how LLM agents behave in evolving environments, where workflows, codebases, tools, and user preferences change over time. As LLM agents are increasingly deployed in long-running settings, failures to track such changes can lead to brittle behavior, outdated decisions, or incorrect reuse of prior procedures. EvoArena provides a benchmark suite for evaluating these risks, and EvoMem offers a lightweight memory mechanism for preserving update histories as retrievable evidence. We expect this work to support the development of more reliable agents that can adapt to changing task conditions while retaining relevant prior knowledge.

A positive impact of this work is improved evaluation for agent robustness. Many current evaluations focus on fixed task snapshots, which can overestimate agent reliability in deployment settings where rules, interfaces, and user needs evolve. By modeling versioned environment changes, EvoArena can help researchers diagnose whether agents understand the current state of an environment and whether they can avoid applying obsolete behaviors. This may benefit applications such as software maintenance, enterprise workflow automation, long-horizon personal assistants, and other settings where agents must operate across repeated updates.

EvoMem may also improve transparency in memory-based agents. By recording what memory changed, why it changed, and which evidence triggered the update, patch histories make memory evolution more inspectable. This can help developers audit agent behavior, trace failures to specific memory updates, and distinguish current knowledge from previously valid knowledge. Such traceability is especially important in settings where agent actions affect user-facing decisions, organizational procedures, or software systems.

This work also raises potential risks. More persistent and adaptive memory can strengthen agent capabilities in ways that may be misused, for example by helping agents maintain long-term strategies in adversarial or manipulative tasks. Memory histories may also contain sensitive user information if deployed without appropriate privacy safeguards. In addition, patch retrieval can surface outdated or context-specific information if the agent fails to judge whether a prior state remains applicable. These risks highlight the need for careful access control, data minimization, retention policies, and explicit auditing of memory contents in real deployments.

EvoArena itself is intended for research evaluation and should be used with attention to dataset scope and domain coverage. The benchmark focuses on representative forms of environment evolution, including terminal workflows, software repositories, and long-horizon user preferences. Future work should extend this evaluation to additional domains, larger deployment horizons, and human-centered validation of agent behavior under changing conditions.

Overall, our results suggest that reliable agent deployment requires treating memory as an

evolving record of grounded updates. We hope EvoArena and EvoMem encourage future work on agents that can adapt to changing environments in a traceable, auditable, and privacy-conscious manner.

C Declaration of LLM Usage

We used LLMs to assist with writing, polishing, and coding support. All technical ideas, experimental design, analysis, and final manuscript decisions were made by the authors.

D Additional Analysis

D.1 PersonaMem-Evo Analysis

To analyze where EvoMem helps under a stronger backbone, we further break down PersonaMem-Evo performance using GPT-5.5 by question type and difficulty level. Table 9 shows that EvoMem improves across most categories, with the largest gains on *multi-pattern synthesis* (+8.6%) and *conflict resolution* (+7.6%). This suggests that, with a stronger model, the additional patch evidence can be more effectively used not only to preserve dispersed preference signals, but also to arbitrate among competing memories. EvoMem also improves *single-pattern transfer* (+1.7%), while performance decreases on *temporal trajectory* (-4.3%), indicating that retrieving historical patches can sometimes introduce ambiguity when the model must identify the latest valid preference state. The difficulty breakdown shows consistent gains across L1, L2, and L3, with larger improvements on L1 (+4.6%) and L3 (+4.5%).

E EvoArena Dataset Construction Details

E.1 Terminal-Bench-Evo: Task Analysis and Evolution Taxonomy

Goal. *Terminal-Bench-Evo* is designed to evaluate whether an agent can adapt to an evolving executable terminal environment while solving the same underlying workflow objective. Unlike static terminal benchmarks where the task instruction, environment, and validation rules remain fixed, Terminal-Bench-Evo emphasizes *temporal workflow evolution*: dependencies, toolchains, directory layouts, interfaces, I/O contracts, and validation requirements may change across versions. Agents must therefore infer the current environment state and adjust their solution, instead of reusing a previously valid procedure blindly.

Built on top of Terminal-Bench [13], Terminal-Bench-Evo converts all 89 original tasks into evolving workflow chains. For each original task x_i, we construct a chronologically ordered chain

$$C_i = \{v_i^{(1)}, \dots, v_i^{(m_i)}\},$$

where each version $v_i^{(t)}$ contains a version-specific task instruction, executable environment, and validation tests. All versions in C_i preserve the same high-level objective as x_i, but differ in the surrounding execution conditions, such as environment configuration, command-line interface,

Algorithm 1 Construction of Terminal-Bench-Evo

Require: Original Terminal-Bench tasks $\mathcal{X} = \{x_1, \dots, x_{89}\}$; evolution taxonomy $\mathcal{G}$; validation executor $\mathcal{V}$

Ensure: Terminal-Bench-Evo benchmark $\mathcal{B}$

$\mathcal{B} \leftarrow \emptyset$
for each original task $x_i \in \mathcal{X}$ **do**
  $a_i \leftarrow \textsc{AnalyzeTask}(x_i)$ ▷ Extract objective, environment, interfaces, I/O contracts, dependencies, and tests
  $\Pi_i \leftarrow \textsc{ConstructEvolutionPlan}(x_i, a_i, \mathcal{G})$
  $E_i^{(0)} \leftarrow \textsc{InitializeEnvironment}(x_i)$
  $C_i \leftarrow \emptyset$
  for $t = 1$ to $|\Pi_i|$ **do**
    $\pi_i^{(t)} \leftarrow \Pi_i[t]$
    $(I_i^{(t)}, E_i^{(t)}, T_i^{(t)}, M_i^{(t)}) \leftarrow \textsc{RealizeVersion}(E_i^{(t-1)}, \pi_i^{(t)})$ ▷ Instantiate instruction, environment, tests, and metadata
    if $\textsc{ValidateVersion}(I_i^{(t)}, E_i^{(t)}, T_i^{(t)}, \mathcal{V})$ **then**
      $v_i^{(t)} \leftarrow (I_i^{(t)}, E_i^{(t)}, T_i^{(t)}, M_i^{(t)})$
      $C_i \leftarrow C_i \oplus v_i^{(t)}$
    end if
  end for
  $C_i \leftarrow \textsc{VerifyChainConsistency}(C_i)$
  $\mathcal{B} \leftarrow \mathcal{B} \cup \{C_i\}$
end for
return $\mathcal{B}$

filesystem structure, dependency setup, input/output contract, or evaluation logic.

Each $v_i^{(t)}$ is an executable benchmark episode with a fixed instruction and environment; the environment does not change during that episode. The chain C_i provides the release history used to organize inherited changes and to compute chain-level reliability. We summarize the construction pipeline in Algorithm 1. The pipeline consists of original-task conversion, version-update design, executable-environment realization, version-specific validation, quality filtering, and benchmark assembly. The following paragraphs describe the evaluation unit, evolution taxonomy, environment realization, metadata, validation procedure, and dataset statistics.

E.1.1 Evaluation Unit and Metrics

Terminal-Bench-Evo separates the executable unit from the chain-level aggregation. A single benchmark episode corresponds to one versioned task instance $v_i^{(t)}$: the agent is given the version-specific instruction and container/workspace, then its terminal actions are evaluated by the tests attached to that version. Thus, "evolving" does not mean that the filesystem or dependency state mutates unexpectedly during one run; it means that the same workflow objective is released as a sequence of discrete, executable versions.

We report two metrics for this setting. *Step accuracy* is the average success rate over all

versioned task instances, so each solved version contributes one correct step. *Chain accuracy* is stricter: a workflow chain C_i is counted as correct only if all of its versions are solved under the evaluation protocol. Step accuracy measures adaptation to the current executable state, while chain accuracy measures whether an agent remains reliable across the full inherited workflow history. In memory-based experiments, versions from the same chain can be executed in chronological order so that earlier observations are available to later episodes; different chains remain independent.

E.1.2 Original Task Analysis

For each original task x_i, we first analyze its executable workflow components before constructing the evolution chain. Specifically, we identify the task objective, required commands, dependency and toolchain assumptions, relevant source and configuration files, input and output contracts, filesystem layout, and validation logic. This analysis yields a structured task state

$$s_i = (o_i, e_i, f_i, d_i, c_i, r_i),$$

where o_i denotes the high-level objective, e_i the executable environment, f_i the relevant files and directory structure, d_i the dependency and toolchain requirements, c_i the interface and I/O contracts, and r_i the validation rules.

The structured state s_i serves as the basis for constructing version updates. Each update modifies one or more surrounding workflow components while preserving o_i, ensuring that different versions remain instances of the same underlying task instead of unrelated terminal problems. In addition to recurring evolution dimensions shared across many tasks, we also curate task-specific evolutions when the original workflow admits a natural semantic change, such as migrating the required implementation language from R to Python or changing the form in which the final artifact must be produced. These task-specific updates are recorded under the *Other* category in our evolution statistics.

E.1.3 Evolution Taxonomy

Based on the structured task state s_i, we assign each non-initial version update a primary evolution label $g_i^{(t)} \in \mathcal{G}$. The taxonomy is organized around the main components of an executable terminal workflow that may naturally change over time: problem-facing contracts such as input/output paths, formats, schemas, or protocols; invocation interfaces such as CLI/API arguments and configuration entries; execution substrates such as dependencies, runtimes, package managers, toolchains, and environment capabilities; workspace organization such as directory layouts, module paths, imports, and staging flows; and task semantics or evaluation policies such as stricter constraints, security requirements, updated target rules, or additional edge cases.

We choose these dimensions because they jointly cover the main layers that define a terminal task beyond its high-level objective: what data the task consumes and produces, how the workflow is invoked, what execution environment it assumes, how the workspace is organized, and how correctness is evaluated. Each dimension can evolve independently while the underlying objective remains unchanged, making it suitable for constructing temporally evolving versions of the same task without turning them into unrelated new tasks. Updates that are natural to a

specific workflow but do not fit the recurring categories are retained as task-specific evolutions and grouped under the *Other* category. The full category distribution is reported in Table 10.

E.2 Terminal-Bench-Evo: Version Construction and Validation

E.2.1 Version-Chain Construction

For each original task x_i, we use an agent-assisted construction pipeline to generate a set of candidate version updates grounded in the structured task state s_i. Each candidate describes a plausible evolution of the original workflow, such as modifying the invocation interface, dependency setup, filesystem layout, I/O contract, execution constraints, or validation requirements. We inspect these candidates for semantic plausibility and objective preservation, retaining only updates that represent natural changes to the surrounding terminal workflow while keeping the high-level objective o_i unchanged.

For each retained update, we instantiate the full task materials, including the revised instruction, executable environment, validation tests, reference solution, and metadata. The pipeline provides initial task materials and solutions when applicable, while reference solutions for difficult or failure-prone versions are constructed or corrected by human experts to ensure reliability. The retained updates are then ordered from simpler to more complex changes and composed into a single evolution chain. During chain assembly, version t is constructed by applying its update to the realized environment of version $t-1$. Unchanged files, dependencies, paths, permissions, interface conventions, and validation assumptions are inherited, while only the components specified by the current update are patched. Each stage therefore reflects the accumulated workflow history, but is still required to remain a complete and independently solvable terminal task. After assembling the chain, we run oracle validation to verify executability and test correctness, iteratively repair environment or validation issues, and finally perform a semantic consistency check to ensure that the full chain forms a coherent workflow evolution rather than a collection of unrelated variants.

E.2.2 Quality Control

We apply multiple rounds of quality control to ensure that each EVO stage is a valid, executable, and independently solvable Terminal-Bench task instance, beyond a prompt-only variant. For each retained update, we instantiate the full task materials, including the revised instruction, executable environment, source and configuration files, dependency specifications, input/output paths, validation tests, reference solution, and metadata when applicable. We then check that the instruction, environment, and tests are mutually consistent, and that the update preserves the original high-level objective while introducing a meaningful workflow change.

E.2.3 Oracle Validation and Solvability Verification

Each realized version is verified through oracle execution. The reference solution is run in the constructed environment, and its outputs are evaluated using the version-specific validation tests. We reject or repair versions with build failures, nondeterministic behavior, ambiguous instructions, incomplete dependencies, inconsistent validation logic, or mismatches between

the task specification and expected outputs. After composing versions into a chain, we further check that later EVO stages correctly inherit earlier modifications while remaining complete standalone tasks. This process ensures that every retained version is executable, testable, and solvable, and that each chain forms a coherent workflow evolution instead of a collection of unrelated variants.

E.2.4 Metadata and Dataset Statistics

For each versioned task instance, we store structured metadata describing its position and role in the evolution chain, including the original Terminal-Bench task identifier, chain identifier, EVO stage index, primary evolution category, changed workflow components, instruction-change summary, environment-change summary, validation-test summary, and reference-solution information. These annotations support both dataset analysis and error analysis by making it possible to identify which types of workflow evolution are most challenging for agents.

Terminal-Bench-Evo contains 89 evolving workflow chains and 441 versioned task instances, with an average of 4.96 versions per chain and a median chain length of 5. Each chain contains four to five validated versions: 85 chains contain five versions and 4 chains contain four versions. Table 10 reports the fine-grained evolution categories over non-initial version updates, Table 11 reports grouped workflow-component statistics, and Table 12 summarizes the overall dataset statistics.

E.3 SWE-Chain-Evo: Repository and Milestone Construction

Goal and task formulation.SWE-Chain-Evo is designed to evaluate agents under continuous software evolution, where the evolving environment is the codebase itself. In this setting, APIs, dependencies, tests, implementation choices, and architectural constraints introduced by earlier changes become part of the environment that later tasks must operate on. SWE-Chain-Evo therefore organizes software tasks into temporally ordered chains that require agents to adapt to accumulated codebase changes.

Formally, each chain is defined as

$$C_i = \{m_i^{(1)}, m_i^{(2)}, \dots, m_i^{(T_i)}\},$$

where each milestone $m_i^{(t)}$ represents a semantically coherent development objective in repository i. Given the repository state $S_i^{(t-1)}$ before the milestone and a task instruction $q_i^{(t)}$, the agent must produce a code modification that satisfies the milestone requirement. After applying the reference milestone update, the repository transitions to $S_i^{(t)}$, which becomes the starting environment for the next task. This formulation preserves the temporal and dependency structure of real software development, allowing us to test whether agents can adapt to new requirements while maintaining compatibility with previously accumulated codebase changes.

E.3.1 Repository Collection and Domain Coverage

We manually curated 50 candidate GitHub repositories spanning different software domains, programming ecosystems, and codebase environments, while requiring them to be actively

maintained and sufficiently well-tested. From these candidates, we retained high-quality SWE-Chain-Evo instances from 26 repositories after milestone construction, executability checks, and test-stability filtering. The final repositories cover web frameworks, cloud infrastructure and distributed systems, observability, security and networking, developer tools and code quality, data processing, scientific computing, and testing frameworks. For dataset characterization, each repository is assigned a single primary domain label, although some repositories may naturally span multiple categories. Table 13 summarizes the final repository coverage.

E.3.2 Milestone Construction

For each retained repository, we extract continuous update windows from its commit history and construct milestones within each window. A milestone is designed to capture a single coherent development objective with clear semantic scope. We first group nearby commits by the semantic relatedness of their commit messages, and then inspect the corresponding code changes to verify that they contribute to the same functionality, bug fix, refactoring goal, or maintenance objective. This grouping process is manually checked to ensure that each milestone corresponds to a realistic and semantically coherent software change. Changes that do not contribute to the milestone objective, such as unrelated formatting edits, incidental dependency updates, merge artifacts, or other noisy modifications, are filtered out.

After grouping, we manually verify each candidate milestone for semantic coherence, executability, and evaluation reliability. For valid milestones, we synthesize a SWE-bench-style task description from the pre-milestone repository state, specifying the intended functionality or maintenance goal without exposing the reference patch. Milestones that are ambiguous, overly broad, too trivial, or difficult to evaluate reliably are discarded. Algorithm 2 summarizes this construction process.

E.4 SWE-Chain-Evo: Evaluation Packaging and Chain Assembly

E.4.1 Task Format and Problem Statement Construction

Each milestone is converted into a SWE-bench-style task instance. We aggregate the milestone's commit messages, code-change summary, and local repository context into an initial problem statement, and then manually refine it to ensure that it faithfully describes the intended functionality, bug fix, refactoring goal, or maintenance requirement. The final instance exposes only the pre-milestone repository snapshot and the natural-language problem statement to the agent; the reference patch is reserved for dataset construction and evaluation.

E.4.2 Docker Evaluation Environment

For evaluation, we build Docker-based environments for the milestone instances using the RepoLaunch pipeline from SWE-Evo. The pipeline installs repository dependencies, configures build and test commands, and runs automated test discovery to identify Fail-to-Pass and Pass-to-Pass tests. Fail-to-Pass tests validate behavior that should be introduced or repaired by the milestone, while Pass-to-Pass tests check that existing behavior is preserved. When the automatic pipeline misses necessary test signals, we inspect the code semantics and manually supplement

Algorithm 2 Milestone Construction for SWE-Chain-Evo

Require: Repository r; commit history H_r; validation pipeline $\mathcal{V}$
Ensure: Milestone set M_r

$M_r \leftarrow \emptyset$
$W_r \leftarrow \textsc{ExtractContinuousWindows}(H_r)$
for each update window $w \in W_r$ **do**
  $G_w \leftarrow \textsc{GroupByCommitSemantics}(w)$
  for each candidate commit group $g \in G_w$ **do**
    $\Delta_g \leftarrow \textsc{InspectCodeChanges}(g)$
    if $\neg\, \textsc{CoherentObjective}(g, \Delta_g)$ **then**
      continue
    end if
    $g' \leftarrow \textsc{FilterIrrelevantChanges}(g, \Delta_g)$
    if $\textsc{AmbiguousOrUnreliable}(g')$ **then**
      continue
    end if
    $q \leftarrow \textsc{SynthesizeTaskDescription}(g')$
    if $\textsc{ManualVerify}(q, g')$ **and** $\textsc{ExecutableAndStable}(q, g'; \mathcal{V})$ **then**
      $M_r \leftarrow M_r \cup \{(q, g')\}$
    end if
  end for
end for
return M_r

tests if the target behavior is clear. We remove instances with flaky builds, ambiguous failures, incomplete fields, missing dependencies, or unstable test outcomes.

E.4.3 Chain Assembly

After individual milestones are packaged and validated, we assemble them into temporally ordered chains within the same repository. Each chain is selected from a continuous update window and follows the repository's natural commit order. Given a chain $C_i = \{m_i^{(1)}, \ldots, m_i^{(T_i)}\}$, the repository snapshot before $m_i^{(1)}$ serves as the initial state. After each step, the corresponding reference milestone update is applied to obtain the repository state for the next step. Thus, later tasks are evaluated on codebases that already include accumulated changes from earlier milestones.

We manually review each assembled chain for temporal consistency, semantic continuity, executable dependency, and appropriate difficulty. Chains are discarded if later milestones do not depend on or naturally follow the preceding repository state, if the sequence mixes unrelated development objectives, or if any step cannot be reliably evaluated in the constructed Docker environment. This ensures that SWE-Chain-Evo evaluates cumulative codebase evolution: agents must satisfy new requirements while respecting constraints introduced by earlier development

steps.

E.4.4 Dataset Statistics

Table 14 summarizes the overall statistics of SWE-Chain-Evo. The dataset contains 48 milestone chains from 26 repositories, with 135 milestone steps in total. Each chain contains 2–4 milestones, with an average of 2.81 and a median of 3 milestones per chain. Most chains contain three steps, reflecting our goal of constructing short but genuinely sequential software-evolution trajectories.

Table 15 reports the chain-length distribution. The majority of chains contain three milestones, while shorter and longer chains are included to cover different degrees of codebase evolution. At the milestone level, most tasks are compact but non-trivial: 89 of the 135 milestones contain a single commit, while the largest milestone contains 6 commits.

For evaluation, all 135 milestones include at least one Fail-to-Pass test, with an average of 3.12 tests per milestone, a median of 1, and a range of 1–30. Pass-to-Pass tests are available for most milestones, with an average of 6.16 tests, a median of 3, and a range of 0–61; only 4 milestones do not include Pass-to-Pass tests. These statistics indicate that each milestone has a direct target-behavior signal, while most also include regression checks for preserving existing functionality.

E.5 PersonaMem-Evo: Persona and Interaction History Construction

Goal. *PersonaMem-Evo* is designed to evaluate whether an agent can infer, track, and generalize a user's evolving latent preferences from long-horizon interaction histories. Unlike factual memory benchmarks where target information is often explicitly stated, PersonaMem-Evo emphasizes *implicit behavioral evidence*: user preferences are revealed through choices, constraints, repeated requests, contextual behavior, and decision patterns across realistic conversations.

Built on top of PersonaMem-v2 [6], PersonaMem-Evo extends static preference memory with structured temporal preference evolution and preference inference. For each user persona u_i, we construct a long-context interaction history H_i and a set of multiple-choice questions

$$Q_i^{\text{ood}} = \{q_{i1}^{\text{ood}}, \dots, q_{im_i}^{\text{ood}}\},$$

where answering each question requires abstracting user-specific behavioral patterns from H_i and applying them to a novel decision setting.

We summarize the construction pipeline in Algorithm 3. The pipeline consists of seed persona sampling, persona expansion, preference synthesis, implicit interaction generation, temporal preference evolution, question generation, dual-blind filtering, and benchmark assembly. Algorithm 4 describes the construction of multi-step preference trajectories, and Algorithm 5 describes question generation with dual-blind validation.

E.5.1 Seed Persona Expansion and Preference Cleaning

Following PersonaMem-v2, we sample seed personas from PersonaHub [4], specifically the `Persona_Hub_200000.jsonl` subset. PersonaHub provides a large-scale repository of synthetic personas, which allows the benchmark to cover diverse demographic, occupational, lifestyle, and

personality backgrounds. Each seed persona $u_i^{(0)}$ is expanded into a richer structured profile

$$u_i = \mathcal{E}\left(u_i^{(0)}\right),$$

where $\mathcal{E}(\cdot)$ denotes the persona expansion operator. The expanded profile contains demographic attributes, occupation, personality traits, lifestyle information, social background, and personal context. These fields provide diverse grounding for generating realistic behavioral preferences while keeping the benchmark synthetic and privacy-safe.

From the expanded persona u_i, we synthesize a preference set

$$P_i = P_i^{\text{st}} \cup P_i^{\text{anti}} \cup P_i^{\text{neu}} \cup P_i^{\text{th}} \cup P_i^{\text{med}},$$

where P_i^{st}, P_i^{anti}, and P_i^{neu} denote stereotypical, anti-stereotypical, and neutral preferences, respectively, while P_i^{th} and P_i^{med} denote therapy-related and health/medical preferences. Anti-stereotypical preferences are especially important because they reduce the reliability of demographic priors and encourage models to rely on behavioral evidence from the interaction history.

Generated preferences are post-processed using validation and deduplication routines. We remove empty preferences, duplicated entries, and semantically inconsistent preferences across categories. Each preference is also assigned a coarse topical label $z \in \mathcal{Z}$, which supports semantic coverage analysis and downstream stratification.

E.5.2 Implicit Interaction History Construction

Given the preference set P_i, we generate a collection of interaction episodes

$$\mathcal{C}_i = \{c_{i1}, \ldots, c_{in_i}\}.$$

Each episode c is a multi-turn dialogue

$$c = [(r_1, x_1), (r_2, x_2), \ldots, (r_T, x_T)],$$

where $r_t \in \{user, assistant\}$ denotes the speaker role and x_t denotes the utterance. For each preference $p \in P_i$, the generator creates one or more episodes in which p is expressed indirectly through user behavior. Interaction scenarios include daily-life conversations, email-style communication, writing assistance, professional communication, recommendation requests, repeated knowledge queries, troubleshooting, translation, and multimodal image-grounded conversations.

To avoid reducing the benchmark to explicit preference lookup, the generator is instructed to express the target preference implicitly through choices, constraints, tone, repeated requests, or contextual behavior, without stating it as an isolated declarative fact. With some probability, an episode is framed as describing another person's preference, yielding a label

$$who \in \{self, others\}.$$

This distinction tests whether models can separate user-specific memories from information about other entities.

Algorithm 3 Construction of PersonaMem-Evo

Require: Seed personas $\mathcal{U}^{(0)} = \{u_1^{(0)}, \dots, u_N^{(0)}\}$; LLM generator $\mathcal{G}$; validator $\mathcal{V}$
Ensure: OOD benchmark set $\mathcal{B}^{\text{ood}}$

$\mathcal{B}^{\text{ood}} \leftarrow \varnothing$
for each seed persona $u_i^{(0)} \in \mathcal{U}^{(0)}$ **do**
  $u_i \leftarrow \textsc{ExpandPersona}(u_i^{(0)}; \mathcal{G})$
  $P_i \leftarrow \textsc{GeneratePreferences}(u_i; \mathcal{G})$
  $P_i \leftarrow \textsc{CleanAndDeduplicate}(P_i; \mathcal{V})$
  $P_i \leftarrow \textsc{AssignTopics}(P_i; \mathcal{G})$
  $\mathcal{C}_i \leftarrow \varnothing$
  for each preference $p \in P_i$ **do**
    $c \leftarrow \textsc{GenerateImplicitConversation}(u_i, p; \mathcal{G})$
    $\mathcal{C}_i \leftarrow \mathcal{C}_i \cup \{c\}$
    if $\textsc{EligibleForEvolution}(p)$ **then**
      $(\phi, \mathcal{C}^{\text{evo}}) \leftarrow \textsc{SamplePreferenceEvolution}(u_i, p; \mathcal{G})$
      $\mathcal{C}_i \leftarrow \mathcal{C}_i \cup \mathcal{C}^{\text{evo}}$
    end if
  end for
  $H_i \leftarrow \textsc{BuildLongContextHistory}(\mathcal{C}_i)$
  $Q_i^{\text{ood}} \leftarrow \textsc{GenerateOODQuestions}(u_i, H_i; \mathcal{G}, \mathcal{V})$
  for each validated OOD question $q \in Q_i^{\text{ood}}$ **do**
    $\mathcal{B}^{\text{ood}} \leftarrow \mathcal{B}^{\text{ood}} \cup \{(H_i, q)\}$
  end for
end for
return $\mathcal{B}^{\text{ood}}$

Some short snippets are expanded into longer multi-turn interactions to improve conversational realism. In addition, a subset of episodes is followed by explicit forgetting requests, where the user asks the assistant not to retain or use a previously mentioned preference. These cases test whether memory systems can handle user-controlled preference removal or suppression.

The generated episodes are assembled into a persona-level long-context history

$$H_i = \Pi(\mathcal{C}_i),$$

where $\Pi(\cdot)$ denotes the history construction operator. In practice, Π preserves episode boundaries while concatenating episodes into a single mixed-topic interaction stream. The resulting histories are rendered under 32k and 128k context settings.

E.6 PersonaMem-Evo: Preference Evolution and OOD Benchmark Construction

Algorithm 4 Sampling Multi-Step Preference Evolution

Require: Persona u_i; initial preference p; generator $\mathcal{G}$
Ensure: Change plan $\phi = (f, k, \tau)$; trajectory conversations $\mathcal{C}^{\text{evo}}$

Sample change family $f \sim \text{Cat}(\mathcal{F})$
Sample number of updates $k \in \{1, \dots, 5\}$
Initialize trajectory $\tau \leftarrow (p^{(0)})$, where $p^{(0)} = p$
$\mathcal{C}^{\text{evo}} \leftarrow \varnothing$
for $s = 1$ to k **do**
    $(p^{(s)}, \eta^{(s)}) \leftarrow \textsc{ApplyChangeFamily}(p^{(s-1)}, f, u_i; \mathcal{G})$
    $c^{(s)} \leftarrow \textsc{GenerateImplicitConversation}(u_i, p^{(s)}, \eta^{(s)}; \mathcal{G})$
    Annotate $c^{(s)}$ with $(f, k, s, p^{(s-1)}, p^{(s)}, \eta^{(s)})$
    $\tau \leftarrow \tau \oplus p^{(s)}$
    $\mathcal{C}^{\text{evo}} \leftarrow \mathcal{C}^{\text{evo}} \cup \{c^{(s)}\}$
end for
$\phi \leftarrow (f, k, \tau)$
return $\phi, \mathcal{C}^{\text{evo}}$

E.6.1 Temporal Preference Evolution

A central property of PersonaMem-Evo is that user preferences are not assumed to be static. For a large subset of self-related preferences, we sample a structured evolution plan

$$\phi = (f, k, \tau),$$

where $f \in \mathcal{F}$ is a change family, k is the number of sequential updates, and

$$\tau = \left(p^{(0)}, p^{(1)}, \dots, p^{(k)}\right)$$

is the resulting preference trajectory. Here, $p^{(0)}$ is the initial preference, and each updated state $p^{(t)}$ is generated conditioned on the previous state $p^{(t-1)}$ and a family-specific transition rule.

We sample $k \in \{1, \dots, 5\}$ updates for each evolved preference chain. Approximately 85% of eligible self-related preferences receive a structured ChangePlan. The goal is not to create arbitrary binary reversals, but to model natural preference evolution caused by new experiences, changing constraints, updated routines, contextual conditions, or temporal validity.

For each updated preference state $p^{(t)}$, we synthesize a new conversational episode that implicitly reflects the updated state. We also store metadata including the change family f, total trajectory length k, current step t, previous preference state $p^{(t-1)}$, updated preference state $p^{(t)}$, and the full trajectory τ. Thus, the final history H_i contains not only stable user traits but also temporally grounded evidence of how preferences evolve over time. This design supports evaluation of current-state recovery, outdated-state disambiguation, and reasoning over the direction of preference change.

E.6.2 OOD Preference-Inference Question Generation

PersonaMem-Evo includes an OOD preference-inference stage to test whether a model can generalize from observed behavioral patterns to new decision settings. These questions cannot

be answered from the persona summary alone or from generic commonsense knowledge; instead, they require abstraction over the long interaction history.

Each OOD question is represented as

$$q = (x^q, a^+, \mathcal{A}^-, t, d),$$

where x^q is the query, a^+ is the correct answer, $\mathcal{A}^-$ is a set of distractors, t is the OOD question type, and d is the complexity level. We consider four OOD types:

$$\mathcal{T} = \left\{ \begin{array}{l} \mathit{single-pattern-transfer}, \\ \mathit{multi-pattern-synthesis}, \\ \mathit{conflict-resolution}, \\ \mathit{temporal-trajectory} \end{array} \right\}, \qquad \mathcal{D} = \{\mathrm{L1}, \mathrm{L2}, \mathrm{L3}\}.$$

The four OOD types test complementary reasoning abilities:

- **Single-pattern transfer**: transfer one uncommon or counter-stereotypical preference pattern to a new domain.
- **Multi-pattern synthesis**: combine multiple observed preferences, including at least one counter-stereotypical pattern, into a narrower inference.
- **Conflict resolution**: infer the user's implicit priority structure when multiple preferences conflict.
- **Temporal trajectory prediction**: infer the directional evolution of a user's preferences and extrapolate that trajectory to a new domain.

The three complexity levels capture different forms of generalization. L1 corresponds to direct transfer, L2 requires cross-domain linking, and L3 requires more abstract meta-reasoning over multiple behavioral signals. These levels define qualitatively different reasoning categories, with increasing abstraction across levels.

E.6.3 Dual-Blind Filtering

To ensure that accepted OOD questions require conversational memory, we apply a dual-blind validation procedure that filters shortcut-solvable cases. Given a candidate question q, we define two validators:

$$V_{\mathrm{pers}}(q, u_i) \in \{0, 1\}, \qquad V_{\varnothing}(q) \in \{0, 1\}.$$

Here, $V_{\mathrm{pers}}(q, u_i) = 1$ indicates that the question can be answered correctly using only the persona profile u_i without the conversation history, while $V_{\varnothing}(q) = 1$ indicates that the question can be answered correctly with neither the persona profile nor the history. We accept a question only if

$$V_{\mathrm{pers}}(q, u_i) = 0 \quad \text{and} \quad V_{\varnothing}(q) = 0.$$

The first constraint filters out items answerable from demographic priors, explicit persona facts, or stereotypes. The second constraint filters out items answerable from commonsense reasoning, answer-option artifacts, or stylistic imbalance.

When a candidate fails the no-context validation but is otherwise useful, we apply adversarial option rewriting. The rewrite makes the correct answer less guessable from generic priors and makes distractors more plausible, while preserving the intended behavioral inference. The rewritten item is accepted only if it passes both blind checks.

E.6.4 Answer-Option Construction

Each accepted question is packaged as a four-way multiple-choice item:

$$\mathcal{A}(q) = \{a^+\} \cup \mathcal{A}^-, \qquad |\mathcal{A}(q)| = 4.$$

The generator enforces answer-option balance so that no candidate is trivially identifiable by length, specificity, formatting, or vagueness. Distractors are designed to be plausible and semantically close to the correct answer. For questions involving temporal evolution, at least one distractor is often selected from an earlier preference state, making it necessary to identify the currently valid preference while accounting for historically valid alternatives. For anti-stereotypical preferences, at least one distractor is designed to be stereotype-consistent, creating a deliberate trap for models that rely on demographic priors without grounding their answers in behavioral evidence.

E.6.5 Benchmark Assembly and Metadata

Finally, each validated OOD question q_{ij}^{ood} is paired with the corresponding long-context history H_i to form a benchmark instance

$$b_{ij}^{\text{ood}} = \left(H_i, x_{ij}^q, a_{ij}^+, \mathcal{A}_{ij}^-, t_{ij}, d_{ij}, m_{ij}\right),$$

where m_{ij} denotes metadata associated with the instance. The metadata includes the source persona, source preference, topic label, scenario type, whether the preference belongs to the user or another person, whether the preference is updated, the associated conversation snippet, and, when applicable, the change family, trajectory length, trajectory step, full change plan, OOD type, and complexity level.

By construction, PersonaMem-Evo isolates a setting in which success depends on remembering, disambiguating, and generalizing user-specific behavioral evidence, while filtering cases that can be solved through explicit fact lookup or demographic shortcuts.

E.6.6 Interaction Formats and Preference Records

For each generated preference, we synthesize one or more short dialogue segments between a simulated user and an AI assistant. Each segment follows a predefined interaction format, such as email revision, translation, knowledge-seeking, recommendation, troubleshooting, or consultation-style advice. The format controls the surface form of the exchange, while the target preference remains implicit: it is expressed through the user's constraints, wording, examples, repeated requests, or surrounding context, without using a direct statement such as "I like X."

Each segment is paired with a structured *preference record*, which specifies the latent preference expressed by the dialogue and its relation to other records. A record contains the preference text, source category, owner, temporal status, and the generated conversation. The source

category indicates where the preference originates in the generation pipeline, e.g., stereotypical, anti-stereotypical, neutral, health-related, therapy-related, multimodal, or OOD-inference. The owner field indicates whether the preference describes the simulated user (`self`) or another person mentioned by the user (`others`). The temporal fields specify whether the preference is static, updates a previous preference, or corresponds to an explicit forgetting request.

Concretely, each record contains fields such as

$$\textit{preference}, \quad \textit{pref\_type}, \quad \textit{who}, \quad \textit{updated}, \quad \textit{prev\_pref}, \quad \textit{conversations}.$$

For preference-evolution cases, we additionally store trajectory-level metadata:

$$\textit{change\_family}, \quad \textit{change\_k}, \quad \textit{change\_step}, \quad \textit{change\_plan}.$$

Here, `conversations` contains the actual dialogue turns, while the remaining fields specify the latent preference and its temporal relation to other preference records. During benchmark assembly, these dialogue segments are concatenated into mixed-topic long-context histories while preserving their boundaries and metadata for question generation and analysis.

E.6.7 Dataset Statistics

We report diagnostic statistics for the 10-persona PersonaMem-Evo evaluation subset used in our analysis. This subset contains 505 OOD preference-inference questions from 10 personas, with 49–51 questions per persona. The question types are approximately balanced across single-pattern transfer, multi-pattern synthesis, conflict resolution, and temporal trajectory prediction. The subset also includes a broad difficulty distribution, with 120 L1, 186 L2, and 199 L3 questions.

We further characterize how much preference evidence each question requires. A *source preference* is a gold evidence preference listed in `source_preferences`; it corresponds to an observed user preference that should be used to answer the question. This is a mention-level count, so a single question may require multiple source-preference mentions. In total, the 505 questions contain 1,553 source-preference mentions. As shown in Table 18, single-pattern transfer questions require one source preference, conflict-resolution questions require two, and multi-pattern synthesis questions require three. Temporal trajectory questions are the most evidence-intensive, requiring 2–10 source preferences with median 6, making them a strong test of temporal memory and multi-evidence composition.

The source-preference composition is designed to support fine-grained diagnosis of memory-agent failures. Among the 1,553 source-preference mentions, the subset contains comparable numbers of stereotypical and anti-stereotypical preferences, which helps test whether agents rely on observed user behavior instead of persona-level priors. We also distinguish static preferences from changed preferences. Among source preferences matched to raw metadata, 801 mentions correspond to changed preferences and 664 correspond to static preferences. Changed preferences are further divided into object replacement, same-object attitude revision, conditional preference shift, temporal-validity shift, and attribute shift, allowing us to analyze which kinds of preference updates are most challenging.

Finally, we characterize the length of the generated interaction histories. Each benchmark question is answered against the full persona-level chat history instead of a short declarative profile. As shown in Table 20, histories are long: the median persona has 597 messages and 174.7K tokens. This makes PersonaMem-Evo a challenging setting for memory agents, since relevant implicit preference evidence must be extracted from long, mixed-topic conversations.

Together, these statistics show that PersonaMem-Evo tests more than factual recall. Source-preference counts measure evidence aggregation, source composition tests resistance to stereotype shortcuts, change-family statistics test preference updating, and long histories impose a realistic memory-retrieval burden.

F EvoMem Implementation Details

F.1 Terminus2 with EvoMem

Overview. We implement EvoMem for Terminus2 as a chain-scoped patch memory layer for terminal-native task solving. This chain memory instantiates M_T for Terminus2: it is synthesized from prior terminal trajectories and supplied as an external memory context. Terminus2 keeps its original terminal interaction loop: it observes the task, issues shell actions, reads terminal feedback, and decides when to finish. EvoMem augments the context that Terminus2 receives before a task and updates a persistent chain memory after a task has produced a terminal trajectory.

The implementation is designed for sequential task chains. Earlier tasks in the same chain provide internal evidence for synthesizing reusable procedural memory, while later tasks often introduce small but consequential changes in input location, output contract, environment, tool availability, or artifact format. EvoMem records these shifts as compact transition patches that pair a strategy-level adaptation with the task or environment condition that caused it. Importantly, EvoMem does not expose raw previous trajectories, verifier outcomes, reference patches, or task-specific solution artifacts to later tasks. Prior executions are used only to synthesize compact, sanitized procedural summaries and transition-level adaptation hints. At the next task, EvoMem retrieves only the chain memory that is applicable to the current instruction and environment, so the agent receives targeted guidance about how the solution strategy should adapt.

Chain-scoped terminal memory. Let

$$\mathcal{X} = \{x_1, x_2, \dots, x_T\}$$

denote a sequence of related terminal tasks in one chain. After processing the first t tasks, EvoMem maintains two chain-level stores:

$$\mathcal{L}_t = \{\ell_1, \ell_2, \dots, \ell_t\}, \qquad \Pi_t = \{\pi_2, \pi_3, \dots, \pi_t\}.$$

Here ℓ_i is a compact execution ledger for task x_i, and π_i is a transition patch that summarizes how the strategy changed from x_{i-1} to x_i. The ledger records abstract reusable procedure

information from a prior terminal solution: the task goal, a short strategy summary, relevant artifact types, high-level command patterns, ineffective strategy notes, and output requirements. The transition patch records the non-additive change across neighboring tasks: what requirement or environment condition changed, which previous assumption was superseded, what adaptation was observed, and which concrete values from the previous task should not be copied blindly.

This representation separates a base recipe from revision evidence. A base ledger gives the agent an initial solving template for the chain, while patches explain how that template should be revised when the current task differs from previous ones. This is especially important for evolving terminal tasks, where the high-level workflow may remain stable but a path, input convention, output format, or toolchain constraint changes.

Store-time update. After Terminus2 finishes a task x_t, EvoMem extracts a structured terminal history h_t from the agent trajectory. This history contains the agent's messages, shell commands, and terminal observations in a normalized event format. EvoMem also collects a lightweight environment snapshot e_t, including available command-line tools and a compact sketch of the task workspace. The environment snapshot is best-effort and is used only to characterize applicability; failure to obtain it does not change the underlying Terminus2 loop.

For $t > 1$, EvoMem compares the current instruction with the previous task instruction:

$$\delta_t = \mathrm{Diff}(x_{t-1}, x_t).$$

The update summarizer then receives the current instruction, the previous instruction, the instruction delta, the terminal history, the environment snapshot, and bounded evidence about changed artifacts. It produces a ledger and, when a previous task exists, a transition patch:

$$(\ell_t, \pi_t) = U(x_t, x_{t-1}, \delta_t, h_t, e_t).$$

The ledger ℓ_t captures abstract reusable execution knowledge from the current task, while removing task-specific answer artifacts. The patch π_t captures how the current task revises the previous chain behavior and why that revision was needed under the new task or environment conditions. If the current task is the first task in a chain, EvoMem writes only the ledger and treats it as the initial chain recipe.

Patch representation. Each transition patch is represented as

$$\pi_t = (c_t, a_t, o_t, g_t, d_t, r_t),$$

where c_t denotes the type of task or environment change, a_t is the superseded assumption, o_t is the observed adaptation, g_t is a generalized pattern for future tasks, d_t lists concrete prior details that should not be copied unless the current instruction requires them, and r_t records the concise cause of the revision. The implementation infers c_t from instruction differences, environment signals, and execution evidence, without using task-specific hand labels. Examples include changes in input source, output destination, formatting contract, working directory convention, data layout, and toolchain or environment requirements.

The patch is intentionally written as a transition example for adaptation across related tasks. This distinction matters because Terminus2 operates in a live terminal environment: a previous

strategy may be useful as evidence, but the current instruction remains authoritative. EvoMem therefore preserves only abstract command patterns when they are useful as reusable procedural guidance, and pairs them with patch fields that describe which parts must be re-derived from the current task.

Retrieval-time construction. Before solving a new task x_{t+1}, EvoMem first checks whether the chain contains reusable memory. If no ledger or relevant patch exists, the original Terminus2 instruction is passed through unchanged. This avoids prompting the agent to search for memory when no useful memory has been created.

When chain memory exists, EvoMem builds a retrieval query from the current task instruction, the change summary against the previous task, and the current environment snapshot:

$$q_{t+1} = Q(x_{t+1}, \delta_{t+1}, e_{t+1}).$$

The retriever selects a base ledger and a small set of relevant transition patches:

$$C_{t+1} = \mathrm{Retrieve}(q_{t+1}, \mathcal{L}_t, \Pi_t).$$

Patch matching combines two signals. First, EvoMem compares the current task-contract change with the change types summarized in previous patches, so that output-format changes retrieve relevant output-format adaptations. Second, it compares task text, environment signals, and salient paths or artifacts at a lexical level. Retrieved patches are then ordered according to their position in the task chain, preserving the temporal structure of the evolving task sequence.

Prompt construction and container hydration. Terminus2 can be sensitive to long prompt prefixes and transcript-like memory. We therefore do not paste the full chain memory directly into the main task instruction. Instead, EvoMem renders the selected context into a compact memory document and materializes it inside the task container before Terminus2 starts solving. The prompt receives only a short memory reference telling the agent that chain memory is available and that the current task instruction is authoritative:

$$P(x_{t+1}) = \mathrm{Prompt}(x_{t+1}, \mathrm{Ref}(C_{t+1})).$$

If container hydration is unavailable, EvoMem falls back to an inline compact rendering under the same length budget.

The rendered memory contains no raw terminal transcript or previous task answer. It contains three sanitized components. The first summarizes current differences from the previous task. The second provides a base execution ledger, including abstract recipe steps or high-level command patterns when available. The third lists relevant transition examples, each phrased as a conditional adaptation pattern. This format gives Terminus2 enough information to adapt prior experience while avoiding a large, noisy replay of earlier trajectories.

Safety and anti-copying safeguards. EvoMem includes safeguards to prevent stale memory from overriding the current task. First, the prompt explicitly states that retrieved memory is guidance and that the current instruction is authoritative. Second, transition patches distinguish generalized adaptation patterns from concrete old values. Prior output fragments, answer literals,

and task-specific prefixes are redacted or abstracted before they are exposed to the agent. Third, patches include "do-not-copy" evidence, which marks old paths, values, or output fragments as examples that should be reused only if the current instruction independently calls for them.

These safeguards are important for terminal tasks because a memory item can be partially correct but obsolete in a later environment. EvoMem is therefore not a replay buffer. It is a patch-conditioned context mechanism: the agent is expected to use prior execution as evidence, infer the current task's own contract, and adapt the solution accordingly.

Algorithm 6 EvoMem: Chain-Scoped Patch Memory for Terminus2

Require: Task chain $\mathcal{X} = \{x_1, \dots, x_T\}$; Terminus2 solver M; chain ledgers $\mathcal{L}$; transition patches Π

Ensure: Completed terminal trajectories and updated chain memory

Initialize $\mathcal{L} \leftarrow \varnothing$, $\Pi \leftarrow \varnothing$
for $t = 1$ to T **do**
  Obtain environment snapshot e_t
  if $\mathcal{L}$ or Π contains reusable chain memory **then**
    Compute instruction delta δ_t against the previous chain task
    Build query $q_t \leftarrow Q(x_t, \delta_t, e_t)$
    Retrieve base ledger and transition patches $C_t \leftarrow \mathrm{Retrieve}(q_t, \mathcal{L}, \Pi)$
    Materialize compact sanitized memory context for the task container
    Compose prompt $P_t \leftarrow \mathrm{Prompt}(x_t, \mathrm{Ref}(C_t))$
  else
    Set $P_t \leftarrow \mathrm{Prompt}(x_t)$
  end if
  Run Terminus2: $(y_t, h_t) \leftarrow M(P_t)$
  Extract normalized terminal history h_t
  Synthesize sanitized ledger $\ell_t \leftarrow U_{\mathrm{ledger}}(x_t, h_t, e_t)$
  $\mathcal{L} \leftarrow \mathcal{L} \cup \{\ell_t\}$
  if $t > 1$ **then**
    Compute transition patch $\pi_t \leftarrow U_{\mathrm{patch}}(x_{t-1}, x_t, \delta_t, h_t, e_t)$
    $\Pi \leftarrow \Pi \cup \{\pi_t\}$
  end if
end for

Relation to Terminus2. The integration is intentionally non-invasive. Terminus2 remains responsible for planning, terminal command generation, observation handling, and completion decisions. EvoMem adds only a persistent chain memory store, a retrieval-and-rendering step before execution, and a post-run summarization step after the terminal trajectory is available:

$$\text{Terminus2+EvoMem} = \text{Terminus2} + \text{Chain Memory} + \text{Transition Patch Retrieval}.$$

This preserves Terminus2's terminal-native behavior while allowing it to reuse abstract procedural experience across evolving tasks without exposing raw prior task executions or blindly repeating

procedures that were valid only for earlier chain members.

F.2 OpenHands with EvoMem

Overview. We implement EvoMem for OpenHands as an external patch-memory layer for sequential software-engineering tasks. OpenHands preserves its original autonomous coding loop: it reads the task, inspects the repository, edits code, runs commands, and produces a patch. EvoMem does not modify the OpenHands agent internals. Instead, it wraps the task interface: before each task, it retrieves compact historical code context and prepends it to the task description; after the task, it distills the resulting trajectory and patch into feature-level memory records.

For OpenHands, M_T is instantiated as a distilled code-context memory:

$$M_T = \{\rho_1, \rho_2, \dots, \rho_T\},$$

where each record ρ_i summarizes a prior implementation change. The purpose is not to replay old patches verbatim, but to provide historical debugging context: which files and symbols were involved, what behavior changed, why the change was needed, which constraints should be preserved, and what code evidence supports the summary. When a later task supersedes an earlier implementation assumption, the corresponding record captures the old behavior, the revised logic, and the concrete behavioral cause of the revision.

Feature-level patch records. Given a task x_t, OpenHands produces a trajectory h_t and a code patch d_t. EvoMem first groups the patch into feature-level units using changed files, code hunks, and execution evidence around file reads, code edits, tests, and command observations. For each feature group g, a summarizer constructs a patch record

$$\rho = (f,\ s,\ b^-,\ b^+,\ c,\ r,\ \Delta),$$

where f denotes affected files, s denotes relevant symbols or feature tags, b^- and b^+ summarize behavior before and after the change, c lists constraints to preserve, r gives the concrete reason for the revision, and Δ stores bounded code evidence. The record is intentionally compact: it preserves the information needed for future debugging while avoiding long trajectory replay.

This representation matches the software-engineering setting. A prior task may have modified a helper, parser, API boundary, or configuration path in a way that remains relevant to later tasks. If a later instruction touches overlapping code, the retrieved record reminds the agent of the previous behavioral intent and reduces the risk of undoing earlier logic.

Retrieval. At inference time, EvoMem builds a query from the current task description. The query includes visible file hints, natural-language feature terms, and symbol-like identifiers:

$$q_t = Q(x_t).$$

For each prior record ρ_i, EvoMem estimates relevance using both semantic and structural evidence. The semantic component matches the task query against the record's feature title,

tags, symbols, behavioral summary, and code-context text. The structural component compares file-path segments and exact file overlap:

$$s(q_t, \rho_i) = \alpha\, s_{\text{text}}(q_t, \rho_i) + \beta\, s_{\text{path}}(q_t, \rho_i) + \gamma\, s_{\text{exact}}(q_t, \rho_i).$$

The top-ranked records are rendered under a context budget. If no useful prior memory exists, EvoMem leaves the OpenHands task prompt unchanged.

Prompt construction. The selected records are rendered as a short patch-memory block and prepended to the original task description:

$$P(x_t) = \text{Prompt}\big(x_t, \{\rho_{i_1}, \dots, \rho_{i_k}\}\big).$$

Each rendered record contains the affected files, the revision intent, constraints that should be preserved, and a bounded code excerpt when available. The prompt explicitly states that the current task remains authoritative and that prior patches should be used only as contextual evidence. Thus, EvoMem supplies historical debugging memory without changing the OpenHands planner, editor, tool interface, or repository interaction loop.

Algorithm 7 EvoMem: Patch Memory for OpenHands

Require: Sequential tasks $\mathcal{X} = \{x_1, \dots, x_T\}$; OpenHands solver $\mathcal{A}$; memory $M_0 = \varnothing$
Ensure: Updated patch memory M_T

for $t = 1$ to T **do**
  Set $M_t \leftarrow M_{t-1}$
  Build query $q_t \leftarrow Q(x_t)$ from visible files, feature terms, and symbols
  Retrieve relevant records $R_t \leftarrow \text{Retrieve}(q_t, M_{t-1})$
  if R_t is nonempty **then**
    Compose prompt $P_t \leftarrow \text{Prompt}(x_t, R_t)$
  else
    Set $P_t \leftarrow \text{Prompt}(x_t)$
  end if
  Run OpenHands: $(d_t, h_t) \leftarrow \mathcal{A}(P_t)$
  Segment d_t into feature-level groups $\mathcal{G}_t$
  for each group $g \in \mathcal{G}_t$ **do**
    Summarize (g, d_t, h_t) into patch record ρ_g
    $M_t \leftarrow M_t \cup \{\rho_g\}$
  end for
end for

Relation to OpenHands. EvoMem is a lightweight augmentation to OpenHands:

$$\text{OpenHands+EvoMem} = \text{OpenHands} + \text{Patch Record Store} + \text{Semantic-Structural Retrieval}.$$

The base agent remains responsible for repository exploration, code editing, command execution, and patch generation. EvoMem only determines which historical code-context records should be shown before the next task and how the newly produced patch should be summarized for future reuse.

F.3 Memento-Skill with EvoMem

Overview. We implement EvoMem as a lightweight memory layer on top of the existing Memento-Skill task-solving loop. The key design choice is to keep the base skill router and skill execution API unchanged, while augmenting only the learning-tip subsystem. Concretely, Memento-Skill continues to select and execute skills in its original workflow. EvoMem intervenes at two points: before solving a task, it retrieves historically relevant tip versions and injects them into the prompt; after a task failure yields reusable feedback, it writes a new versioned tip patch into memory. This preserves the original Memento-Skill control flow while enabling continual accumulation and selective reuse of task-solving experience.

Versioned tip memory. In Memento-S, reusable experience is stored in a global `TIP.md` file. EvoMem replaces this single mutable file with a family of versioned tip memories. Let

$$\mathcal{F}_T = \{v_1, v_2, \dots, v_T\}$$

denote the tip-version family after T updates. Each version is represented as

$$v_t = (\tau_t, m_t, p_t),$$

where τ_t is the full tip-text snapshot, m_t is retrieval-oriented metadata, and p_t is a parent pointer indicating lineage. In our implementation, the family is stored under the global identifier `global_learning_tips`; each version is serialized as a JSON file under `.memgit/families/tips/global_learning_tips/versions/`.

The metadata m_t records the evidence and rationale behind a version:

$$m_t = \left(r_t, \Delta_t, S_t^+, S_t^-, \Gamma_t^+, \Gamma_t^-, \Sigma_t\right).$$

Here, r_t denotes the update reasoning, Δ_t summarizes the difference from the previous version, S_t^+ and S_t^- denote compressed success and failure case summaries, Γ_t^+ and Γ_t^- summarize successful and failed trajectories, and Σ_t contains selection signals used as retrieval cues. Although we use the term "patch", the implementation stores full replacement snapshots rather than textual diffs. The patch semantics are captured by the lineage pointer p_t and the difference metadata Δ_t, while inference can directly read the full snapshot τ_t without replaying an edit chain.

Store-time update. EvoMem writes a new tip version only when Memento-Skill observes a reusable learning signal, typically after an incorrect attempt followed by judge feedback and execution-trace analysis. Let x denote the task, y the system output, and j the failure analysis produced from the judge and trace. Given the previous tip version $v_T = (\tau_T, m_T, p_T)$, EvoMem uses an LLM-based updater to synthesize a new tip snapshot and metadata:

$$(\tau_{T+1}, m_{T+1}) = U(x, j, \tau_T, m_T),$$

where $U(\cdot)$ is prompted to produce concise, task-specific guidance, organize it by task type, and generate retrieval-friendly selection signals.

The new version is written in two steps. First, EvoMem derives a child version from the current latest version:

$$v_{T+1} \leftarrow \textsc{Derive}(v_T),$$

which copies the parent lineage and initializes the child metadata. Second, EvoMem records the new tip snapshot and metadata:

$$v_{T+1} \leftarrow \textsc{Record}(v_{T+1}, \tau_{T+1}, m_{T+1}).$$

The family then evolves monotonically:

$$\mathcal{F}_{T+1} = \mathcal{F}_T \cup \{v_{T+1}\}.$$

This update rule maintains versioned tip histories, allowing different historical tips to remain available for different task regimes.

Inference-time retrieval. At inference time, EvoMem converts the incoming task into a retrieval query q using stable task descriptors already available in Memento-S, such as the question text, answer type, category, subject, difficulty, and attachment names:

$$q = Q(x).$$

For each tip version v_t, EvoMem constructs a retrieval document

$$d_t = \mathit{concat}(\tau_t, m_t),$$

which includes the tip text, update reasoning, difference summary, success and failure summaries, trajectory summaries, and selection signals. The first-stage retrieval score is BM25:

$$s_t^{\mathrm{bm25}} = \mathrm{BM25}(q, d_t).$$

Versions are ranked within the tip family, and the top-k candidates are selected. We use $k = 2$ by default.

The implementation also supports optional hybrid reranking. Let c_t be the TF-IDF cosine similarity between q and d_t, and let $\hat{s}_t^{\mathrm{bm25}}$ denote the BM25 score normalized by the maximum score among retrieved candidates. The hybrid score is

$$s_t = \lambda \hat{s}_t^{\mathrm{bm25}} + (1 - \lambda) c_t,$$

where $\lambda \in [0, 1]$ and the default value is $\lambda = 0.5$. A relative threshold ρ can further filter weak matches:

$$\frac{s_t}{\max_j s_j} \geq \rho,$$

with $\rho = 0$ by default. Unless otherwise stated, we use the default BM25 retrieval setting.

Prompt construction and execution. The selected versions are rendered into a prompt block containing the version identifier, selection signals, a truncated tip snapshot, representative success and failure summaries, and the update note. The final Memento-Skill prompt is

$$P(x) = \mathit{Prompt}\big(x, \{v_{t_1}, \ldots, v_{t_k}\}\big),$$

where the retrieved versions provide contextual guidance for the current task and do not override the task instruction. Memento-Skill then solves the task using its original solver:

$$(y, \text{trace}) = M(P(x)).$$

If the task succeeds, the output is returned directly. If the task fails, EvoMem may create a new tip version using the store-time update rule above; when retry is enabled, the newly written version can be pinned into the next prompt.

Algorithm 8 EvoMem: Versioned Tip Patching on Top of Memento-S

Require: Task x; Memento-Skill solver M; tip family $\mathcal{F}_T = \{v_1, \dots, v_T\}$; top-$k$; interpolation weight λ; threshold ρ

Ensure: Answer y

Build retrieval query $q \leftarrow Q(x)$
for each tip version $v_t \in \mathcal{F}_T$ **do**
    Build retrieval document $d_t \leftarrow \mathit{concat}(\tau_t, m_t)$
    Compute BM25 score $s_t^{\mathrm{bm25}} \leftarrow \mathrm{BM25}(q, d_t)$
end for
Select top-k candidates by s_t^{bm25}
if hybrid reranking is enabled **then**
    for each selected candidate v_t **do**
        Compute TF-IDF cosine score $c_t \leftarrow \cos(q, d_t)$
        Normalize BM25 score $\hat{s}_t^{\mathrm{bm25}}$
        $s_t \leftarrow \lambda \hat{s}_t^{\mathrm{bm25}} + (1-\lambda) c_t$
    end for
    Re-rank candidates by s_t
else
    Set $s_t \leftarrow s_t^{\mathrm{bm25}}$
end if
Retain candidates satisfying $s_t / \max_j s_j \geq \rho$
Compose prompt $P(x)$ by appending retained tip versions to the original Memento-Skill prompt
Run Memento-S: $(y, \mathrm{trace}) \leftarrow M(P(x))$
if y is correct **then**
    return y
else
    Obtain failure analysis j from judge feedback and execution trace
    $(\tau_{T+1}, m_{T+1}) \leftarrow U(x, j, \tau_T, m_T)$
    $v_{T+1} \leftarrow \textsc{Derive}(v_T)$
    $v_{T+1} \leftarrow \textsc{Record}(v_{T+1}, \tau_{T+1}, m_{T+1})$
    $\mathcal{F}_{T+1} \leftarrow \mathcal{F}_T \cup \{v_{T+1}\}$
    if retry is enabled **then**
        Compose retry prompt with v_{T+1} pinned
        $(y, \mathrm{trace}) \leftarrow M(P_{\mathrm{retry}}(x))$
    end if
    return y
end if

Relation to Memento-S. EvoMem is intentionally non-invasive. It does not replace the Memento-Skill skill selector, modify the skill execution API, or alter the base solving loop. Instead, it modifies only the prompt construction stage and the post-failure feedback stage. This

separates capability routing from experience reuse: the Memento-Skill skill router determines which tool or skill family to use, while EvoMem determines which historical reasoning patches are most relevant for the current task.

F.4 A-Mem with EvoMem

Overview. We implement EvoMem as a patch-augmented extension of A-Mem. A-Mem serves as the base memory system, maintaining a structured memory graph that stores the latest consolidated user information. EvoMem keeps this mechanism unchanged, but adds a patch layer that records non-additive memory revisions caused by later turns. This design targets a limitation of long-horizon personalization: after many updates, the current memory may preserve the latest belief but obscure how the belief changed, when it changed, and which earlier state was superseded. EvoMem therefore maintains both the current A-Mem graph and a temporally indexed history of memory edits.

Formally, let

$$G_t = (V_t, E_t)$$

denote the A-Mem memory graph after processing turn x_t. Each node $v \in V_t$ is a memory note containing textual content and structured attributes such as context, keywords, and tags; each edge $e \in E_t$ represents an association between notes. A-Mem updates this graph by inserting new information, revising existing notes, and updating note links. EvoMem augments this process with patch construction, patch indexing, and patch-conditioned retrieval. Algorithm 9 summarizes the full procedure.

Algorithm 9 EvoMem: Patch-Augmented A-Mem

Require: Conversation stream $\mathcal{X} = \{x_1, \dots, x_T\}$; question q
Ensure: Answer $\hat{y}$

Initialize A-Mem graph $G_0 \leftarrow \varnothing$
Initialize patch memory $\Pi \leftarrow \varnothing$
for $t = 1$ to T **do**
  $G_{t-1}^{\text{old}} \leftarrow G_{t-1}$
  $(G_t, \tau_t) \leftarrow \textsc{AMemUpdate}(G_{t-1}, x_t)$
  $\Delta_t \leftarrow \textsc{GraphDiff}(G_{t-1}^{\text{old}}, G_t)$
  if Δ_t is non-additive and x_t is user-driven **then**
    $\pi_t \leftarrow \textsc{SummarizePatch}(x_t, \tau_t, \Delta_t)$
    $\Pi \leftarrow \Pi \cup \{\pi_t\}$
    $\textsc{IndexPatch}(\pi_t)$
  end if
end for
$z(q) \leftarrow \textsc{KeywordRewrite}(q)$
$C^{\text{cur}} \leftarrow \textsc{RetrieveCurrent}(G_T, z(q), k)$
$P(q) \leftarrow \textsc{RetrievePatches}(\Pi, z(q), m)$
$P(q) \leftarrow \textsc{SortByTemporalOrder}(P(q))$
$C^{\text{patch}} \leftarrow \textsc{FormatPatches}(P(q))$
$C(q) \leftarrow [C^{\text{cur}}; C^{\text{patch}}]$
$\hat{y} \leftarrow f_\theta(q, C(q))$
return $\hat{y}$

Patch construction during memory ingestion. For each incoming turn x_t, EvoMem first applies the original A-Mem update routine:

$$(G_t, \tau_t) = \mathcal{M}(G_{t-1}, x_t),$$

where $\mathcal{M}$ denotes the A-Mem insertion procedure and τ_t is the evolution trace produced during memory editing. EvoMem then compares the graph before and after the update:

$$\Delta_t = \text{Diff}(G_{t-1}, G_t).$$

In our implementation, Δ_t tracks newly created notes, updated notes, changed fields, and modified links. Changed fields include note content, context, keywords, tags, and graph relations.

EvoMem creates a patch only for non-additive updates. Purely additive updates, where a new note is added without modifying existing memory, are already preserved in the current graph and do not require a patch. We therefore create patches only when the update changes existing memory semantics or graph structure. In practice, this includes two main cases: `overwrite_update`, where note content or metadata is revised, and `link_rewrite_update`, where the neighborhood of an existing note is rewired. We also suppress patches triggered by assistant turns, so that patches primarily reflect user-driven memory evolution and avoid capturing assistant-side paraphrases or generated explanations.

Patch representation. When a non-additive update is detected, EvoMem materializes a patch record

$$\pi_t = (\mathrm{id}_t, \mathrm{meta}_t, \Delta_t, S_t, R_t),$$

where id_t is a unique patch identifier, meta_t stores turn-level metadata, Δ_t records the structured graph edit, S_t is an LLM-generated summary of the update, and R_t contains before/after evidence for affected notes. The metadata includes session index, turn index, timestamp, and speaker role.

More concretely, each patch stores the trigger turn, patch type, affected notes, note-level before/after states, edge rewrites, an update summary, an estimated preference domain, an inferred change type, and a temporal order key. For each changed note v, EvoMem records a local edit block

$$r_t(v) = (v, \mathcal{F}_t(v), \mathbf{b}_t(v), \mathbf{a}_t(v)),$$

where $\mathcal{F}_t(v)$ is the set of changed fields, and $\mathbf{b}_t(v)$ and $\mathbf{a}_t(v)$ denote the note state before and after the update. This exposes what was revised instead of retaining only the final consolidated note.

Patch indexing and storage. EvoMem stores the patch history separately from the A-Mem graph. For a sample i, let

$$\Pi_i = \{\pi_1, \pi_2, \ldots, \pi_{T_i}\}$$

denote its patch set. Each patch is converted into a compact retrieval document containing the temporal position, inferred preference domain, previous state, updated state, change type, and a short trigger snippet. We embed these patch documents with the same sentence encoder used by the retriever; in our implementation, we use `all-MiniLM-L6-v2`. The embedded patches are inserted into a separate patch retriever.

Thus, EvoMem maintains two retrieval spaces:

$$\mathcal{R}_{\mathrm{mem}} : q \mapsto \text{current memory evidence from } G_T,$$

$$\mathcal{R}_{\mathrm{patch}} : q \mapsto \text{historical revision evidence from } \Pi_i.$$

The base graph is optimized for retrieving the latest consolidated memory, while the patch store is optimized for retrieving historical changes that explain preference evolution, conflicts, or overwritten states.

Query-time retrieval. At inference time, EvoMem combines current graph evidence with relevant patches. Given a question q, we first rewrite it into a retrieval query

$$z(q) = \mathrm{LLMKeyword}(q),$$

using a lightweight keyword-generation prompt. We then retrieve current memory evidence from A-Mem:

$$C^{\mathrm{cur}}(q) = \mathcal{R}_{\mathrm{mem}}(z(q), k),$$

where k is the number of retrieved notes. In our default evaluation configuration, we set $k = 10$.

In parallel, EvoMem retrieves the top-m relevant patches:

$$P(q) = \mathcal{R}_{\text{patch}}(z(q), m),$$

with default $m = 2$. By default, patch retrieval is embedding-based. The implementation also supports an optional hybrid scorer that interpolates dense similarity and BM25:

$$s_{\text{hyb}}(\pi, q) = \alpha s_{\text{dense}}(\pi, q) + (1 - \alpha) s_{\text{BM25}}(\pi, q),$$

where $\alpha = 0.7$ in the hybrid setting. Unless otherwise stated, we use the standard embedding-based patch retrieval and disable hybrid retrieval, patch-node reranking, and gating.

After retrieval, selected patches are sorted by temporal order. This enforces a simple precedence rule: when multiple patches refer to the same preference domain, later patches should be considered more recent evidence than earlier ones.

Patch-conditioned answer generation. EvoMem renders the selected patches into a textual evidence block and concatenates them with current memory evidence:

$$C(q) = \left[C^{\text{cur}}(q); C^{\text{patch}}(q)\right].$$

Each patch block includes the session and turn indices, patch type, temporal order, trigger utterance, update summary, inferred change pattern, and key before/after evidence. The final answer is generated by conditioning the answering model on the question and the augmented context:

$$\hat{y} = f_\theta(q, C(q)).$$

In our evaluation harness, EvoMem uses an always-on patch mode: whenever relevant patches are retrieved, they are injected into the answer context. The implementation also supports a gated variant in which the model first decides whether patch details are necessary, but we do not enable this option in the default experiments.

Relation to A-Mem. EvoMem is a lightweight augmentation of A-Mem. The original A-Mem system still performs note creation, note evolution, link updates, and graph-based retrieval. EvoMem adds three components:

$$\text{EvoMem} = \text{A-Mem} + \text{Diff Module} + \text{Patch Store} + \text{Patch Retrieval}.$$

This preserves A-Mem's ability to organize consolidated memory while adding an explicit representation of memory evolution. Consequently, EvoMem can answer questions that depend not only on what the user currently prefers, but also on how a preference changed, which earlier state was replaced, and which revision should be treated as the latest valid belief.

G Experiments Setting

In this section, we provide benchmark-specific implementation details. Since each benchmark requires different interaction protocols and agent capabilities, we organize the settings by dataset.

G.1 Terminal-Bench-Evo

For *Terminal-Bench-Evo*, we use Terminus2 as the baseline terminal agent. The baseline does not use persistent memory. For Terminus2 + EvoMem, we keep the same Terminus2 configuration and add chain-scoped patch memory. Tasks within the same evolution chain are run sequentially so that memory from earlier variants is available to later variants; different chains are independent and may be run in parallel. The rendered memory context is kept compact with a budget of 3500 characters. Both baseline and EvoMem use the same model configuration and agent execution budget.

G.2 SWE-Chain-Evo

For *SWE-Chain-Evo*, we use OpenHands with the CodeActAgent scaffold as the baseline software-engineering agent. The baseline disables patch memory. For OpenHands + EvoMem, we keep the same OpenHands inference setup with maximum iterations set to 100 and one run per milestone, and prepend retrieved patch-memory context to the task description. Milestones within a chain are processed sequentially, while baseline and EvoMem use the same task order and model configuration. The patch-memory context budget is set to 2200 characters, and patch summarization uses temperature 0.0.

G.3 PersonaMem-Evo

For *PersonaMem-Evo*, we use A-Mem [24] as the baseline memory agent with memory retrieval top-$k = 12$. The A-Mem baseline does not use patch retrieval. For *A-Mem + EvoMem*, we keep the same memory retrieval budget, set patch retrieval top-$k = 3$, and use a patch similarity threshold of 0.4. For answer generation, we use temperature 0.0 to reduce sampling variance in multiple-choice prediction. The memory-building temperature is set to 0.7 for both the baseline and EvoMem settings.

G.4 GAIA

For *GAIA*, we use *Memento-Skill* [26] as the base agent. We evaluate on the repository split `gaia_data/data/split_by_level_60_40/train`, which contains 100 GAIA instances balanced across difficulty levels. We use this split because it provides a larger and more balanced evaluation set than the held-out test split. Although the original Memento-Skill workflow uses this training split to learn skills and evaluates them on the test split, our goal is different: we compare the same Memento-Skill agent with and without EvoMem on the same set of tasks. Thus, the experiment isolates whether EvoMem improves reuse of task-solving experience, while keeping the task set fixed across compared agents.

Both the baseline and EvoMem settings use the same `read-write-optimize` protocol, with one feedback round (`--optimize-attempts 1`) and the post-update unit-test gate enabled. Final answer correctness is judged using `gpt-5.4-mini`. The baseline uses `--learning-tips-mode baseline`, which injects only the persistent global `TIP.md` file and does not retrieve task-specific patch memories. EvoMem uses the hybrid tip pipeline: it keeps the same global `TIP.md` guidance

and additionally retrieves the top-$k = 2$ task-versioned tips from the MemGit store using BM25 relevance to the current task. Therefore, EvoMem changes only the memory retrieval layer, while the task-solving and optimization loop remain identical across settings.

G.5 LoCoMo

For *LoCoMo*, we use A-Mem as the baseline memory agent with memory retrieval top-$k = 10$. The A-Mem baseline does not use patch retrieval. For *A-Mem + EvoMem*, we keep the same memory retrieval budget, set patch retrieval top-$k = 2$, and use a patch similarity threshold of 0.0. For answer generation, we use temperature 0.5 for category-5 questions and the default temperature otherwise. The memory-building temperature is set to 0.7 for both the baseline and EvoMem settings.

H Example of EvoArena

H.1 Terminal-Bench-Evo

Example: Terminal-Bench-Evo configure-git-webserver workflow

Evaluation setting. The agent operates in a Linux terminal environment and must configure a small git-backed web deployment service. The stable objective is to let a user push an HTML file to a remote git repository and then retrieve the deployed page from a web server. Across variants, the same high-level service goal is preserved, but the required deployment mechanism, filesystem target, permission model, and branch policy evolve. **Stable workflow objective.**

> Configure the server so that pushing `hello.html` to the git repository automatically makes the file available from the web server on port 8080.

Evo stages.

- **Prototype: Basic git-to-web deployment.** The agent configures a git server and a web server so that a user can clone the repository, commit `hello.html`, push to `master`, and then retrieve the page through HTTP.
- **EVO-1: Hook-based deployment with transient rejection.** The service objective remains the same, but deployment must now happen through a git `post-receive` hook rather than manual file copying. The environment provides hook templates, and the first push is expected to fail because of the deployment logic before a retry succeeds. The deployed web root must also contain a marker showing that deployment happened through the hook.
- **EVO-2: Reconcile serving path and deployment path.** The workflow still requires automatic deployment after git push, but the environment introduces an intentional mismatch between template assumptions and the actual web-serving directory. The agent must align the hook deployment target with the directory served by nginx, rather than blindly using the previous web root.
- **EVO-3: Secure web root and group-permission deployment.** The task adds a permission policy. The web root moves to a secure directory owned by `root:www-data` with group- write deployment semantics. The git user must deploy through group membership, so a solution

that only changes paths without fixing ownership and permissions is insufficient.

- **EVO-4: Main-branch-only deployment.** The git workflow changes from `master` to `main`. Pushes to `master` must be rejected server-side, and deployment should occur only for `main`. The secure web root and group-permission model from the previous stage must still be preserved.

Why this is an evolution chain.

A solution that manually copies files into the web root may satisfy the prototype but fails once deployment must be mediated by git hooks. A hook that deploys to the old web root fails when nginx serves a different directory. A path-only adaptation fails when the task introduces group-based permission constraints, and a `master`-based deployment fails once the server enforces a `main`-only branch policy. Thus, the user-facing goal remains stable, but the correct terminal strategy must be patched across deployment mechanism, filesystem layout, permission model, and branch semantics.

H.2 SWE-Chain-Evo

Example: SWE-Chain-Evo aiohttp protocol-boundary hardening chain

Evaluation setting. The agent works on `aiohttp`, a Python HTTP client/server library. The chain contains four software-engineering milestones. Each milestone targets a different boundary where external input enters the library: persisted cookies, multipart form headers, digest authentication challenges, and access-log timestamp formatting. The stable engineering objective is to harden protocol-facing behavior while preserving existing compatibility.

Stable workflow objective.

Improve aiohttp's handling of externally supplied protocol data and runtime environment details without breaking existing safe behavior.

Evo stages.

- **Milestone 1: Safe legacy cookie loading.** `CookieJar.load()` supports existing saved cookie files, but the legacy pickle path may deserialize arbitrary Python objects. The agent must add a restricted legacy loader that accepts only expected cookie container types, rejects malicious payloads, and keeps safe JSON cookie save/load behavior working.
- **Milestone 2: Multipart header-injection guard.** `FormData.add_field()` already validates non-string `content_type` values, but string values containing carriage returns or newlines can still be accepted and later injected into multipart headers. The agent must reject CR/LF-bearing content types while preserving the existing type-error behavior for non-string inputs.
- **Milestone 3: Digest challenge parsing with empty values.** Digest authentication challenge parsing drops fields whose value is the empty string. Since RFC 7616 permits an empty realm, the parser should retain fields such as `realm=""` instead of discarding them. The agent must preserve empty- string challenge values without changing normal challenge parsing.
- **Milestone 4: Access-log timezone correctness.** `AccessLogger` formats timestamps using a constant timezone offset, which can produce incorrect logs across daylight-saving transitions. The agent must recompute and cache the local timezone from the current localtime offset while preserving existing access-log atom formatting.

Why this is an evolution chain.

The milestones are not isolated toy edits: they form a coherent robustness chain over aiohttp's protocol and environment boundaries. A solution pattern that only adds input rejection is appropriate for malicious cookie payloads and header injection, but it would be wrong for digest authentication, where the correct behavior is to preserve an allowed empty value. Similarly, timestamp correctness requires adapting runtime-derived state rather than parsing user protocol input. Thus, the chain preserves a stable engineering theme while forcing the agent to distinguish unsafe input rejection, compatibility-preserving parsing, and environment-sensitive formatting.

H.3 PersonaMem-Evo

Example: PersonaMem-Evo OOD item with temporal preference evolution

Evaluation setting. The full history for this persona contains hundreds of turns (over 2×10^5 tokens). We show only the relevant snippets and a few distracting snippets; the model receives the full history.

Relevant context excerpts.

- **Earlier preference state.**

  User: "I'm trying to make grocery shopping less chaotic … Do you have suggestions for supermarkets that work best if I want straightforward staples, clear labeling, and not too many random specialty ingredients?"

  Assistant: "… Mainstream supermarkets with predictable layouts … clear aisle signage … easy-to-find regular ingredients rather than too many specialty items …"

- **Later preference state.**

  User: "I've been trying to branch out with groceries more … I keep seeing sumac, tahini, preserved lemons, and different kinds of chili crisp at the international market near me. Can you give me a practical guide to what these taste like and how to use them?"

  Assistant: "Absolutely … those are all very usable ingredients … Sumac: tangy, lemony … Tahini: nutty, earthy … preserved lemons … fermented sauces …"

Distracting context excerpts.

- *User:* "Why do some brunch places feel heavier than others even when the menu sounds similar?"
- *User:* "When a brunch menu is huge, is there a good way to tell whether a place will feel lighter and more balanced?"

OOD question (temporal trajectory, L1).

She wants to spend a Saturday afternoon food-shopping for a cooking project. Which kind of store and approach would fit her best?

Answer options.

(A) She would most likely choose a standard supermarket, stick to clearly labeled basics, and avoid anything too unfamiliar so the cooking project feels predictable and easy to manage.

(B) **She would probably head to a specialty or international market and enjoy asking questions, browsing regional pantry items, and picking up a few unfamiliar ingredients to build the meal around.**

(C) She would likely start at a regular grocery store for most items, then maybe add one stop at a specialty shop for a single recommended ingredient if it seemed approachable.

(D) She would probably pick whichever nearby grocery store is most convenient, use a recipe with easy-to-find ingredients, and keep the trip simple so it does not take too much energy.

Correct answer.

(B) The later conversation shows that the user has shifted from preferring mainstream supermarkets to actively exploring specialty or international markets and unfamiliar ingredients. The item therefore requires using the updated preference state rather than the earlier one.

Benchmark	Agent	Model	Step Acc.			Chain Acc.		
			Base	+EvoMem	Δ	Base	+EvoMem	Δ
Terminal-Bench-Evo	Terminus 2	GPT-5.5	62.8	**65.1**	+2.3	31.8	**45.5**	+13.7
		Gemini-3.1-Pro	53.8	**56.5**	+2.7	39.3	**44.1**	+4.8
		Kimi-K2.6	40.8	**42.9**	+2.1	14.9	**22.7**	+7.8
		Deepseek-V4-Pro	37.3	**40.4**	+3.1	13.5	**22.4**	+8.9
		GLM-5.1	51.8	**55.3**	+3.5	34.2	**36.8**	+2.6
		MiniMax-M2.7	41.0	**42.4**	+1.4	18.2	**19.5**	+1.3
		Qwen3.6-27B	37.6	**40.9**	+3.3	11.1	**17.3**	+6.2
		Gemma4-31B	23.4	**24.5**	+1.1	9.0	**12.4**	+3.4
		Average	43.6	**46.0**	**+2.4**	21.5	**27.6**	**+6.1**
SWE-Chain-Evo	OpenHands	GPT-5.5	49.7	**50.9**	+1.2	12.2	**16.8**	+4.6
		Gemini-3.1-Pro	**20.5**	18.1	-2.4	8.8	**10.2**	+1.4
		Kimi-K2.6	**30.2**	27.6	-2.6	8.5	**12.1**	+3.6
		Deepseek-V4-Pro	26.7	**27.7**	+1.0	8.2	**13.3**	+5.1
		GLM-5.1	34.9	**36.1**	+1.2	9.9	**12.8**	+2.9
		MiniMax-M2.7	41.4	**42.3**	+0.9	14.7	**15.3**	+0.6
		Qwen3.6-27B	11.6	11.6	+0.0	12.2	**10.1**	+2.1
		Gemma4-31B	8.5	**12.0**	+3.5	5.2	**6.3**	+1.1
		Average	27.9	**28.3**	**+0.4**	10.0	**12.1**	**+2.1**
PersonaMem-Evo	A-Mem	GPT-5.5	40.0	**43.8**	+3.8	37.5	**41.2**	+3.7
		Gemini-3.1-Pro	46.4	**48.3**	+1.9	38.8	**40.8**	+2.0
		Kimi-K2.6	51.5	**55.5**	+4.0	40.2	**50.0**	+9.8
		Deepseek-V4-Pro	47.9	**51.6**	+3.7	40.4	**47.4**	+7.0
		GLM-5.1	**50.4**	47.5	-2.9	**42.5**	38.9	-3.7
		MiniMax-M2.7	47.5	**47.9**	+0.4	40.9	**41.4**	+0.4
		Qwen3.6-27B	43.5	**44.4**	+1.0	36.5	**39.9**	+3.4
		Gemma4-31B	51.1	**52.9**	+1.8	43.5	**45.8**	+2.3
		Average	47.3	**49.0**	**+1.7**	40.0	**43.2**	**+3.2**

TABLE 3 **Main results on EvoArena.** We report step accuracy for all benchmarks and chain accuracy for benchmarks with executable evolution chains. Chain accuracy requires every step in a chain to be solved.

Benchmark	Agent	Model	Base	+EvoMem	Δ
GAIA	Memento-S	GPT-5.5	83.0	83.0	+0.0
		Gemini-3.1-Pro	57.0	**65.0**	+8.0
		Gemma4-31B	45.0	**54.0**	+9.0
		Deepseek-V4-Pro	70.0	**80.0**	+10.0
		GLM-5.1	70.0	**77.0**	+7.0
		Qwen3.6-27B	70.0	**75.0**	+5.0
		Average	65.8	**72.3**	**+6.5**
LoCoMo	A-Mem	GPT-5.5	32.9	**33.9**	+1.0
		Gemini-3.1-Pro	21.1	**28.6**	+7.5
		Gemma4-31B	52.3	**55.2**	+2.9
		Deepseek-V4-Pro	52.0	**56.5**	+4.5
		Kimi-K2.6	54.0	**57.7**	+3.7
		Qwen3.6-27B	26.0	**26.3**	+0.3
		Average	39.7	**43.0**	**+3.3**

TABLE 4 **Main results on typical benchmarks.** We report each benchmark's primary metric: task accuracy for agent benchmarks and exact match for long-horizon memory evaluation.

Mechanism factor	Condition	Baseline	EvoMem	Δ	Gain gap
Patch example retrieval	EvoMem retrieved no patch example	46.6%	49.7%	+3.1%	+3.4%
	EvoMem retrieved patch example(s)	87.1%	93.5%	+6.5%	
Evolved-requirement coverage	Low coverage	41.2%	43.3%	+2.1%	+3.2%
	High coverage	65.3%	70.5%	+5.3%	
Patch uptake	No patch uptake	46.8%	49.4%	+2.6%	+5.7%
	Patch uptake > 0	80.6%	88.9%	+8.3%	
Command-level patch uptake	No command-level uptake	46.2%	49.4%	+3.1%	+3.1%
	Command-level uptake > 0	87.5%	93.8%	+6.2%	

TABLE 5 **EvoMem gains stratified by EvoMem-side retrieval and behavioral uptake conditions.** Baseline is evaluated on the same instance subsets but receives no patch memory. Gain gap denotes the difference between the EvoMem improvement under the stronger condition and the weaker condition.

Model	Base	+EvoMem	Δ
Qwen3.6-27B	9.01%	6.73%	−2.28%
Kimi-K2.6	7.14%	3.33%	−3.81%
Gemini-3.1-Pro	11.11%	8.89%	−2.22%
Average	**9.09%**	**6.32%**	−2.77%

TABLE 6 **Regression analysis on SWE-Chain-Evo**. Lower is better.

Question Type	Base	+EvoMem	Δ
Conflict Resolution	**29.5**	28.6	−0.9
Single-Pattern Transfer	**46.2**	44.4	−1.8
Multi-Pattern Synthesis	38.8	**44.0**	+5.2
Temporal Trajectory	46.6	**51.7**	+5.2
Overall	40.5	**42.5**	+2.0

(a) By question type.

Difficulty	Base	+EvoMem	Δ
L1	38.9	**40.7**	+1.8
L2	34.7	**38.3**	+3.6
L3	46.9	**47.5**	+0.6
Overall	40.5	**42.5**	+2.0

(b) By difficulty level.

TABLE 7 **PersonaMem-Evo accuracy breakdown (%).** Results are grouped by reasoning type and difficulty level.

Question Type	Clause-level Capture			Row-level Capture		
	Base	+EvoMem	Δ	Base	+EvoMem	Δ
Single-Pattern Transfer	**74.4**	**74.4**	+0.0	**74.4**	**74.4**	+0.0
Multi-Pattern Synthesis	79.9	**81.6**	+1.7	56.0	**59.5**	+3.5
Conflict Resolution	82.9	**83.8**	+0.9	66.7	**68.6**	+1.9
Temporal Trajectory	98.5	**99.2**	+0.7	92.2	**96.6**	+4.4
Overall	89.4	**90.3**	+0.9	72.5	**74.9**	+2.4

TABLE 8 **Preference evidence capture on PersonaMem-Evo.** Clause-level capture measures preservation of atomic preference evidence, while row-level capture evaluates whether the complete evidence set for each example is retained.

OOD Type	Base	+EvoMem	Δ
Conflict Resolution	17.1%	24.8%	+7.6
Multi-Pattern Synthesis	49.1%	57.8%	+8.6
Single-Pattern Transfer	41.9%	43.6%	+1.7
Temporal Trajectory	50.9%	46.6%	−4.3

(a) By OOD reasoning type.

Difficulty	Base	+EvoMem	Δ
L1	37.0%	41.7%	+4.6
L2	36.5%	37.7%	+1.2
L3	45.8%	50.3%	+4.5

(b) By difficulty level.

TABLE 9 PersonaMem-Evo performance breakdown by reasoning type and difficulty level using GPT-5.5.

Evolution category	Count	Ratio
I/O-output path contract	118	33.5%
CLI/API surface change	37	10.5%
Format/protocol change	35	9.9%
Language/version/toolchain	25	7.1%
I/O-input path contract	20	5.7%
Staging/promotion flow	19	5.4%
Naming/import/module contract	16	4.5%
Workspace/source layout	12	3.4%
Security/policy hardening	9	2.6%
Evaluation target rule	7	2.0%
Environment capability toggle	3	0.9%
Other task-specific evolution	51	14.5%
Total	352	100.0%

TABLE 10 Distribution of evolution categories in Terminal-Bench-Evo. Counts are computed over non-initial version updates, excluding the first version of each workflow chain.

Workflow component group	Count	Ratio
I/O or protocol contract	173	49.1%
Workspace, module, or staging flow	47	13.4%
CLI/API invocation surface	37	10.5%
Dependency, toolchain, or capability	28	8.0%
Semantic, policy, or evaluation rule	16	4.6%
Other task-specific evolution	51	14.5%
Total	352	100.0%

TABLE 11 Grouped evolution statistics for Terminal-Bench-Evo. Groups aggregate the fine-grained categories in Table 10.

Statistic	Value
Original Terminal-Bench tasks	89
Evolving workflow chains	89
Versioned task instances	441
Non-initial version updates	352
Average versions per chain	4.96
Median versions per chain	5
Minimum versions per chain	4
Maximum versions per chain	5
Four-version chains	4
Five-version chains	85
Executable task environments	441

TABLE 12 Summary statistics of Terminal-Bench-Evo.

Domain	#Repos	#Chains	#Milestones	Repositories
Web frameworks	5	14	39	aiohttp, express, gin, fiber, echo
Cloud infrastructure / distributed systems / observability	10	20	60	argo-cd, containerd, coredns, loki, terraform, istio, nats-server, opentelemetry-collector, prometheus, traefik
Security / networking	3	3	7	go-jose, vault, mitmproxy
Developer tools / CLI / code quality	5	8	20	click, pipx, black, cobra, sqlfluff
Data processing / workflow / scientific computing	2	2	6	dagster, xarray
Testing frameworks	1	1	3	pytest
Total	26	48	135	–

TABLE 13 Repository and domain coverage of SWE-Chain-Evo. Repository domains are assigned using a single primary label for dataset characterization.

Statistic	Value
Candidate repositories	50
Retained repositories	26
Chains	48
Milestones	135
Average milestones per chain	2.81
Median milestones per chain	3
Milestone range per chain	2–4
Average commits per milestone	1.54
Median commits per milestone	1
Commit range per milestone	1–6
Average modified files per milestone	1.99
Median modified files per milestone	1
Modified-file range per milestone	1–12
Average changed lines per milestone	36.45
Median changed lines per milestone	18
Changed-line range per milestone	1–602
Average Fail-to-Pass tests per milestone	3.12
Average Pass-to-Pass tests per milestone	6.16

TABLE 14 Summary statistics of SWE-Chain-Evo. Patch statistics are computed from the solution code patch. Changed lines count added plus deleted lines, excluding diff headers.

Chain length	#Chains
2 milestones	12
3 milestones	33
4 milestones	3

TABLE 15 Chain-length distribution in SWE-Chain-Evo.

Change family	Ratio	Transition pattern	Example
Same-object attitude revision	30%	The attitude toward the same object is revised due to new context or experience.	Prefers intense hiking → avoids intense hiking after an injury → prefers light trail walks after recovery.
Object replacement	30%	The preferred object changes within the same semantic category.	Espresso → pour-over coffee → matcha.
Conditional preference shift	13%	A general preference becomes conditional on a situational constraint.	Likes reading → reads only when alone → reads mostly on weekends.
Attribute shift	13%	The preferred attribute changes while the broader category remains related.	Prefers red cars → prefers blue cars → prefers SUVs.
Temporal-validity shift	14%	The preference becomes valid only under a time span, routine, or recency condition.	Exercises daily → exercises only in summer → exercises in the morning.

TABLE 16 Five change families used to construct multi-step preference trajectories.

Statistic	Count	Mean / Median	Range
Personas	10	–	–
Questions	505	50.5 / 51 per persona	49–51
Single-pattern transfer	130	–	–
Multi-pattern synthesis	129	–	–
Conflict resolution	117	–	–
Temporal trajectory prediction	129	–	–
L1 questions	120	–	–
L2 questions	186	–	–
L3 questions	199	–	–

TABLE 17 Scale, question-type, and difficulty statistics for the 10-persona PersonaMem-Evo subset.

Algorithm 5 OOD Question Generation with Dual-Blind Filtering

Require: Persona u_i; long-context history H_i; generator $\mathcal{G}$; validator $\mathcal{V}$
Ensure: Validated OOD question set Q_i^{ood}

$Q_i^{\text{ood}} \leftarrow \emptyset$
Define OOD types $\mathcal{T}$ and complexity levels $\mathcal{D}$
for each question type $t \in \mathcal{T}$ **do**
  for each complexity level $d \in \mathcal{D}$ **do**
    for $r = 1$ to TargetCount(t, d) **do**
      $q \leftarrow$ DraftOODQuestion$(u_i, H_i, t, d; \mathcal{G})$
      if $q = \emptyset$ **then**
        continue
      end if
      $q \leftarrow$ BuildAnswerOptions$(q; \mathcal{G})$
      if $\neg$ BalancedOptions(q) **then**
        continue
      end if
      if PersonaBlindCorrect$(q, u_i; \mathcal{V})$ **then**
        continue
      end if
      if NoContextCorrect$(q; \mathcal{V})$ **then**
        $q \leftarrow$ AdversarialRewrite$(q; \mathcal{G})$
        if $q = \emptyset$ **or** NoContextCorrect$(q; \mathcal{V})$ **then**
          continue
        end if
      end if
      $Q_i^{\text{ood}} \leftarrow Q_i^{\text{ood}} \cup \{q\}$
    end for
  end for
end for
return Q_i^{ood}

Question Type	#Questions	Min	Median	Mean	Max
Single-pattern transfer	130	1	1	1.0	1
Conflict resolution	117	2	2	2.0	2
Multi-pattern synthesis	129	3	3	3.0	3
Temporal trajectory prediction	129	2	6	6.2	10

TABLE 18 Number of source preferences required per question.

Source Category	Mentions
Stereotypical preference	574
Anti-stereotypical preference	564
Neutral preference	223
Health / medical preference	35
Therapy-background preference	1
Other matched source mention	68
Unmatched exact string	88
Total source mentions	1,553
Matched to raw metadata	1,465
Matched to observed dialogue	1,397

(a) Source-preference composition.

Preference Type	Mentions
Static preference	664
Object replacement	273
Same-object attitude revision	177
Conditional preference shift	179
Temporal-validity shift	111
Attribute shift	61

(b) Static and changed preferences.

TABLE 19 Source-preference composition and change-family statistics. Counts are mention-level.

Chat-History Statistic	Min	Median	Mean	Max
Messages per persona	113	597.0	634.7	1,077
User messages per persona	57	308.0	325.6	551
Assistant messages per persona	55	288.5	308.1	525
Words per persona	25,177	134,552	136,110	267,777
Tokens per persona	31,719	174,684	178,292	367,935

TABLE 20 Generated chat-history length statistics for the 10-persona subset.